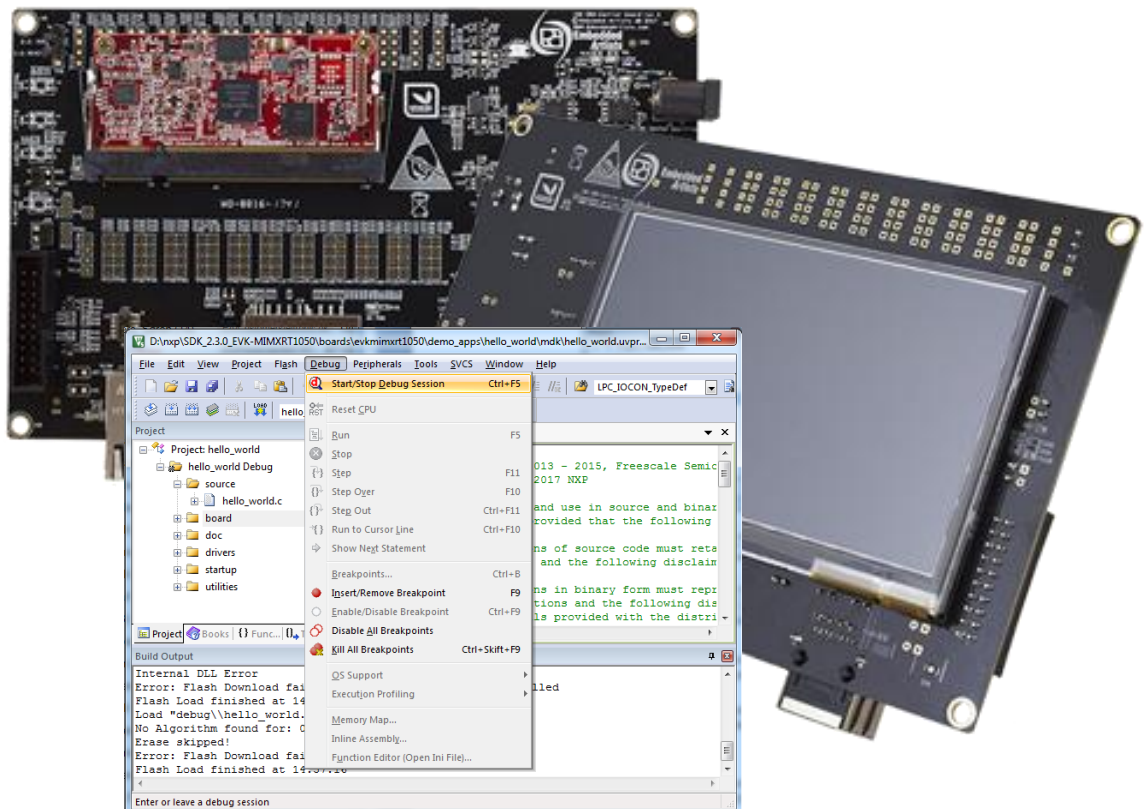


iMX RT1052/1062 Developer's Kit Program Development Guide



*Get Up-and-Running Quickly and
Start Developing Your Application On Day 1!*

Embedded Artists AB

Jörgen Ankersgatan 12
211 45 Malmö
Sweden

<http://www.EmbeddedArtists.com>

Copyright 2020 © Embedded Artists AB. All rights reserved.

No part of this publication may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language or computer language, in any form or by any means, electronic, mechanical, magnetic, optical, chemical, manual or otherwise, without the prior written permission of Embedded Artists AB.

Disclaimer

Embedded Artists AB makes no representation or warranties with respect to the contents hereof and specifically disclaim any implied warranties or merchantability or fitness for any particular purpose. Information in this publication is subject to change without notice and does not represent a commitment on the part of Embedded Artists AB.

Feedback

We appreciate any feedback you may have for improvements on this document. Please send your comments to support@EmbeddedArtists.com.

Trademarks

All brand and product names mentioned herein are trademarks, services marks, registered trademarks, or registered service marks of their respective owners and should be treated as such.

Table of Contents

1	Document Revision History	5
2	Get Started with Program Development	6
2.1	Downloading and Installing MCUXpresso SDK from NXP	6
2.2	Modify the SDK to Support iMX RT Developer's Kit	11
2.3	SDK Project Comments	11
2.3.1	Common for All Examples	11
2.3.2	USB Examples	13
2.3.3	Network Examples	13
2.3.4	Wi-Fi Examples	13
2.3.5	Bluetooth Low Energy (BLE) Example	14
2.3.6	Bluetooth Passthrough Example	15
2.3.7	Crank Storyboard CoffeeApp Example	16
2.3.8	Examples using SD Card (only applicable to iMX RT1062 Developer's Kit)	16
2.4	Getting Started With a Specific IDE	17
3	Getting Started with Keil uVision/MDK and SDK	18
3.1	Install CMSIS Device Pack	18
3.2	Build an Example Application	18
3.3	Run an Example Application	19
3.4	Selecting Build Target	21
3.5	Configuring Flash Algorithm	22
4	Getting Started with Keil uVision/MDK and RTE	23
4.1	Install CMSIS Device Pack	23
4.2	Build an Example Application	24
4.3	Run an Example Application	25
4.4	Selecting Build Target	27
4.5	Configuring Flash Algorithm	28
4.6	Working with RTE	29
5	Getting Started with IAR Embedded Workbench	30
5.1	Build an Example Application	30
5.2	Run an Example Application	32
5.3	Configuring Flash Algorithm	33
6	Getting Started with NXP MCUXpresso IDE	35
6.1	Install the SDK	35
6.2	Build an Example Application	35
6.3	Run an Example Application	39
7	Debug Interface	40
7.1	J-LINK/J-TRACE Support	41

7.1.1	Install J-LINK Software	41
7.1.2	MCUXpresso 10.3.0	41
7.1.3	Keil uVision v5.25.2	42
7.2	Debug Troubleshooting	44
8	Standalone Program Download	45
8.1	Install and Patch the Required Software	45
8.2	Prepare the Program to Flash	46
8.3	Bootting an Unsigned Image	46
8.4	Bootting an Authenticated or Encrypted Image	47
9	Terminal Application Setup	48
9.1	UART-to-USB Bridge	48
9.2	Terminal Application on the PC	48
9.2.1	Tera Term Terminal Emulation Application	49
9.2.2	PuTTY terminal emulation application	50
10	Bootting from External Memory	51
11	Things to Note	54
11.1	ESD Precaution	54
11.2	General Handling Care	54
11.3	OTP Fuse Programming	54
11.4	Further Information	55
12	Disclaimers	56
12.1	Definition of Document Status	57

1 Document Revision History

<i>Revision</i>	<i>Date</i>	<i>Description</i>
A	2020-01-31	- Updated to match SDK 2.7.0 - Added information about MCUXpresso Secure Provisioning Tools - Updated example descriptions and patches
A1	2020-05-13	- Updated information in section 2.3.1 regarding clocking and the new patched file

2 Get Started with Program Development

This chapter contains information about how to get started with program development on the *iMX RT1052 Developer's Kit* and the *iMX RT1062 Developer's Kit*. Since the kits are almost identical from a program development perspective, for the rest of this document the name *iMX RT Developer's Kit* will be used. Only when there is a difference will the individual *iMX RT Developer's Kit* be referenced.

The document also contains information about program download, i.e., how to flash the EcoXiP flash memory.

To start program development you need the following things, all of them:

1. **MCUXpresso SDK** - this is a package of sample software that must be downloaded from NXP's website. You need to create an account on NXP's website for this.
2. **Apply a zip-file** to the SDK with projects specific to the *iMX RT Developer's Kit*. The zip-file is downloaded from the *iMX RT Developer's Kit* support page on Embedded Artists' website.
3. **Integrated Development Environment (IDE)**
 - a. As of August 2018, **NXP MCUXpresso, Keil uVision/MDK and IAR Embedded Workbench** supports direct flash programming of the EcoXiP. Update to latest version of respective IDE.
 - b. Programming of the EcoXiP flash is also supported via NXP's Mfgtool standalone application but it is not a suitable tool to use during program development.
4. **JTAG probe** to be able to download the application to SRAM, SDRAM or the EcoXiP flash memory and in general to be able to debug - set breakpoints, inspect memory, etc.
 - a. The low-cost LPC-Link2 is an excellent choice. Keil ULINK2 and ULINKplus, as well as Segger JLINK, are also excellent debug probes.
 - b. Technically it is possible to program/flash the EcoXiP without a JTAG probe (via NXP's Mfgtool application), but it is strongly recommended to use the proper tool for debugging - i.e., use a JTAG probe!
5. And of course the ***iMX RT Developer's Kit!***

2.1 Downloading and Installing MCUXpresso SDK from NXP

This section will walk you through the installation of the MCUXpresso SDK, which is a package of sample software projects (with device drivers and peripheral examples and demos) that will get you started immediately with your i.MX RT software development. Note that even though the name of the SDK suggests the code package is for the MCUXpresso IDE, the sample projects have project files for other IDEs as well, such as Keil uVision/MDK, IAR Embedded Workbench, and more.

First download the MCUXpresso SDK by following this URL: <https://mcuxpresso.nxp.com/en/welcome>. This is the online SDK builder that makes sure you will get the latest version of the software.

Note that some details in the screen dumps in this section might change over time, but the basic walkthrough steps are the same.

You will need to login to your NXP account. Then click on *Select Development Board*.

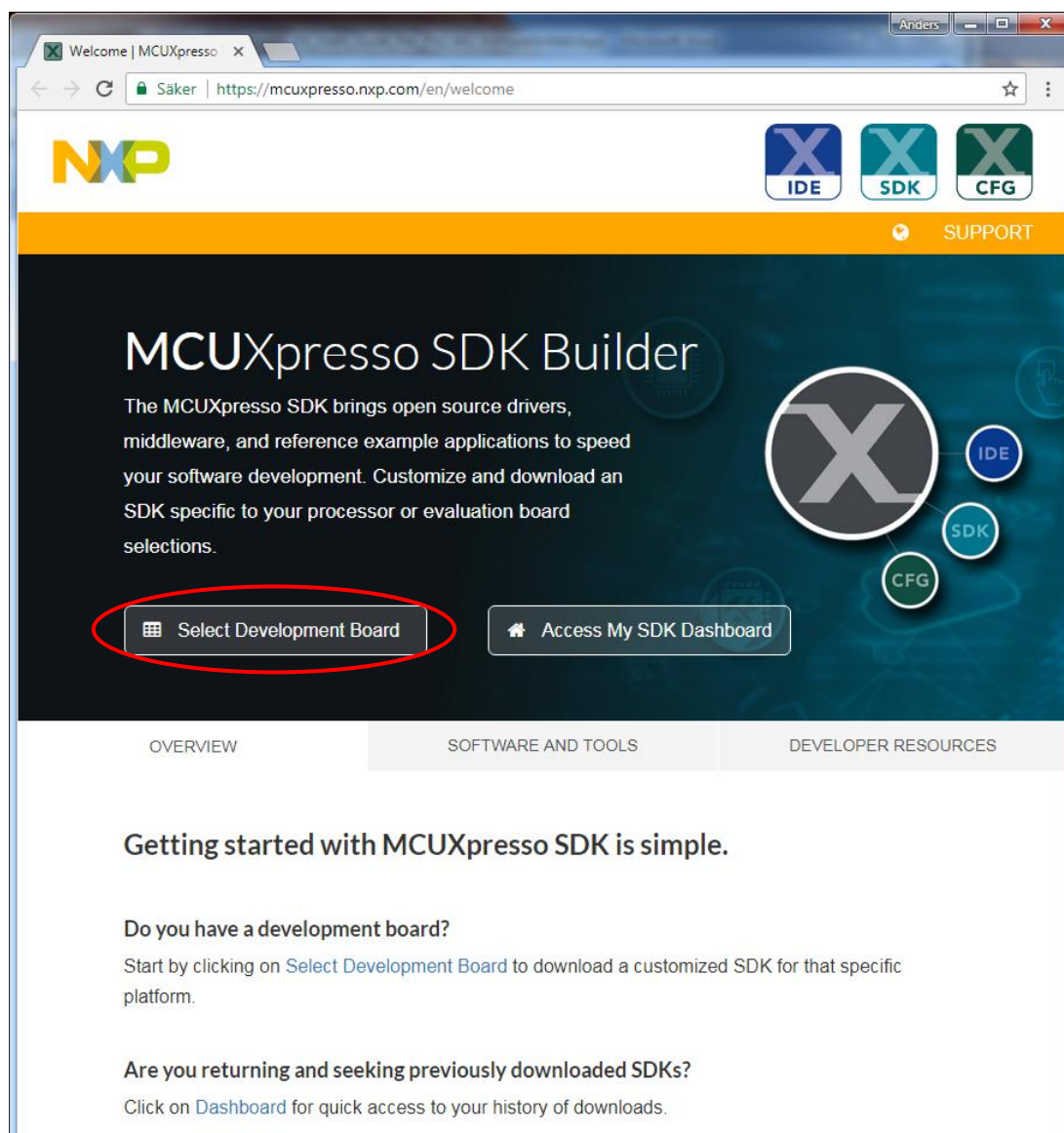


Figure 1 – MCUXpresso SDK Builder

Then select the Boards → i.MX → EVKB-IMXRT1050 or EVK-MIMXRT1060 (depending on if you are using the iMX RT1052 or RT1062 Developer's Kit). This is the NXP evaluation board, but most of the samples will work unmodified on the *iMX RT Developer's Kit*.

Note that there are two different versions of the MIMXRT1050 EVK - EVK and EVKB. It is the latter that shall be selected because that supports i.MX RT1052 silicon revision A1.

For i.MX RT1062, only one version of the NXP EVK exists.

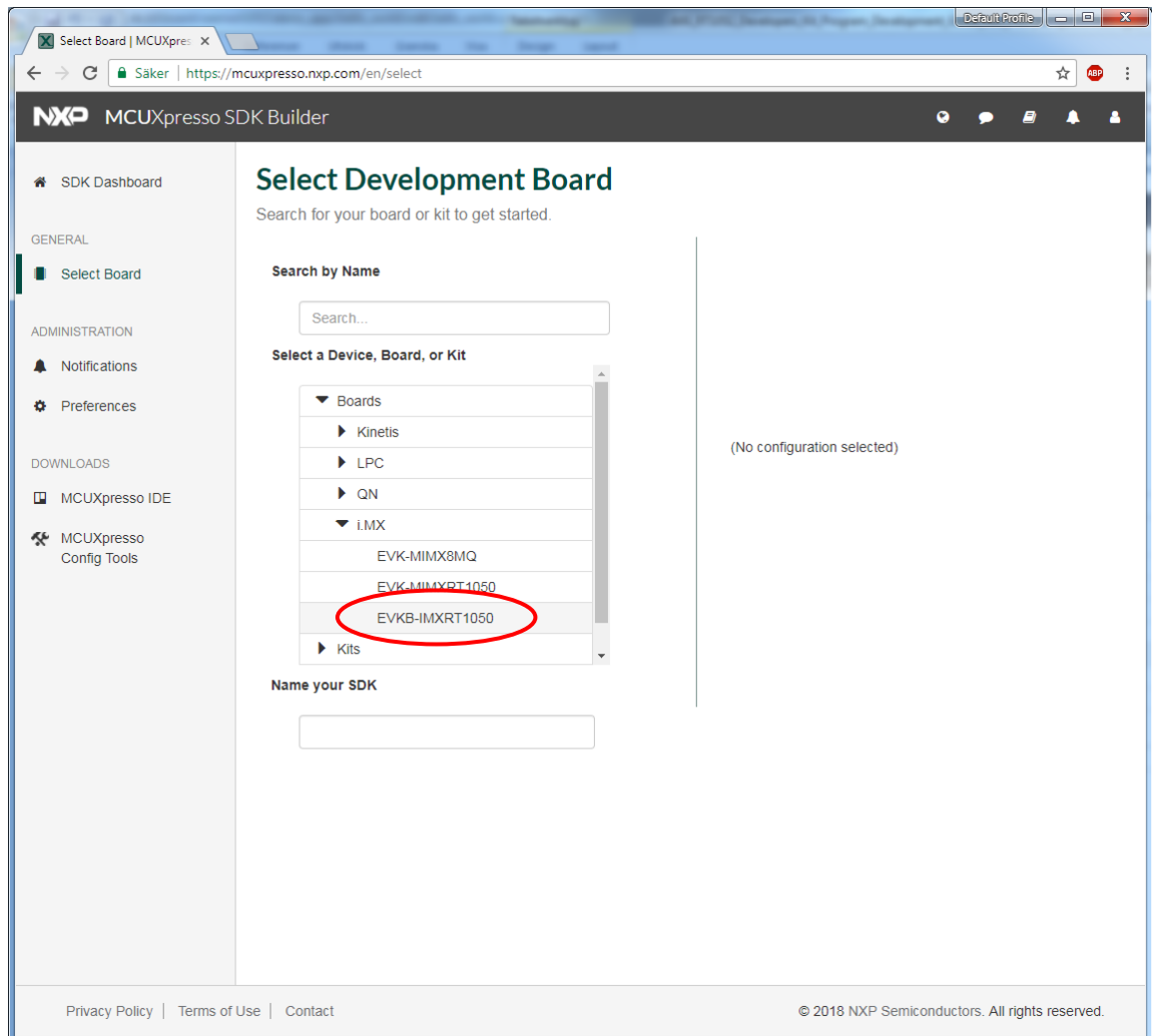


Figure 2 – MCUXpresso SDK Builder - Select Development Board

After selecting the NXP EVK, click on the *Build MCUXpresso SDK* button.

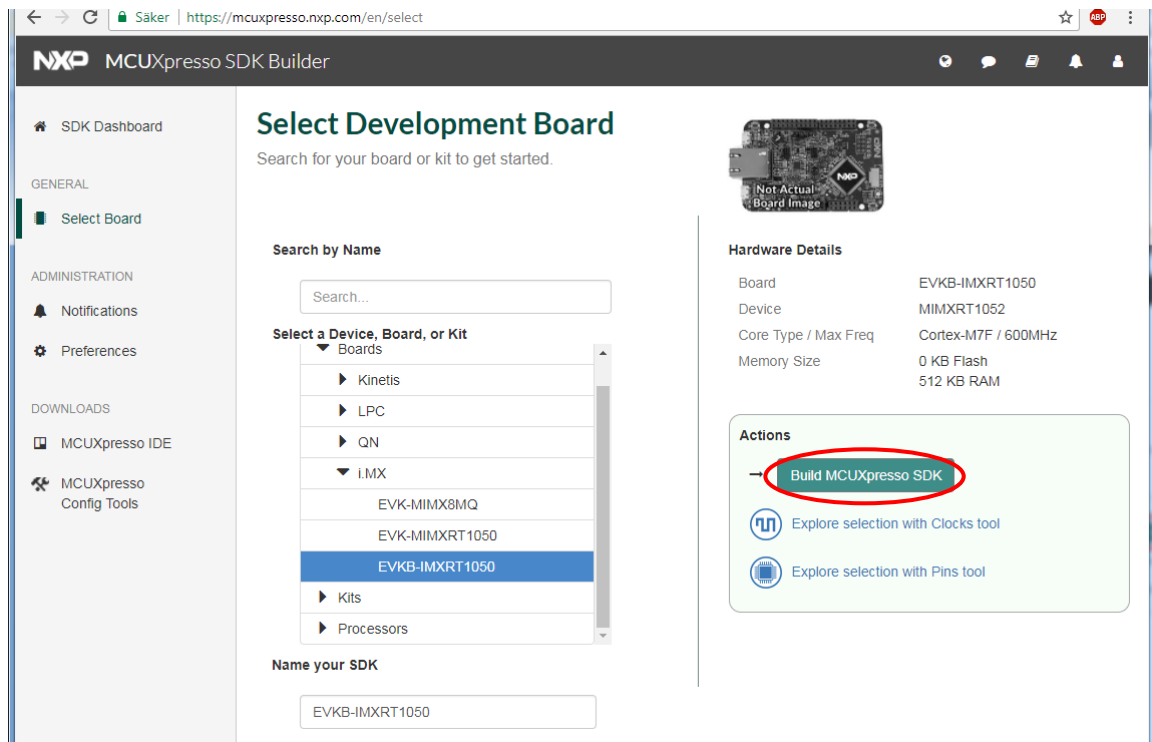


Figure 3 – MCUXpresso SDK Builder - Build MCUXpresso SDK

Select *All Toolchains* under the *Toolchain/IDE* dropdown list. Note the option to select a specific version of the SDK.

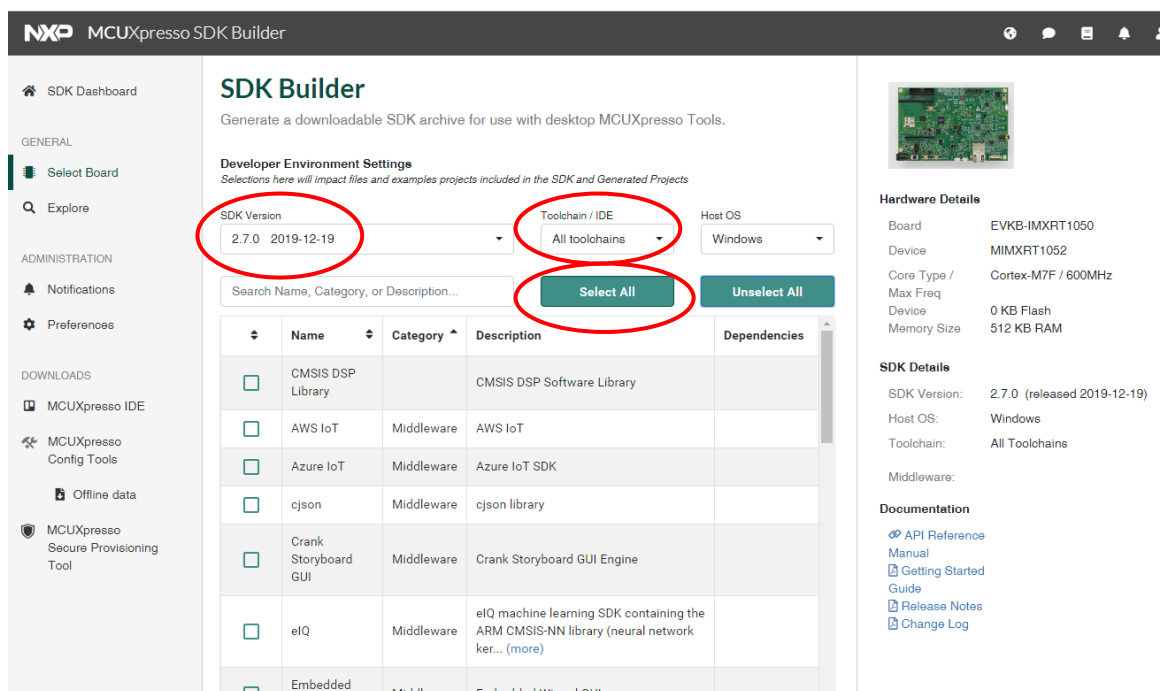


Figure 4 – MCUXpresso SDK Builder - Select Toolchain

Click the *Select All* button to select all optional packages.

Scroll to the bottom of the page and press the *Download SDK* button. This will give you a complete package with all software.

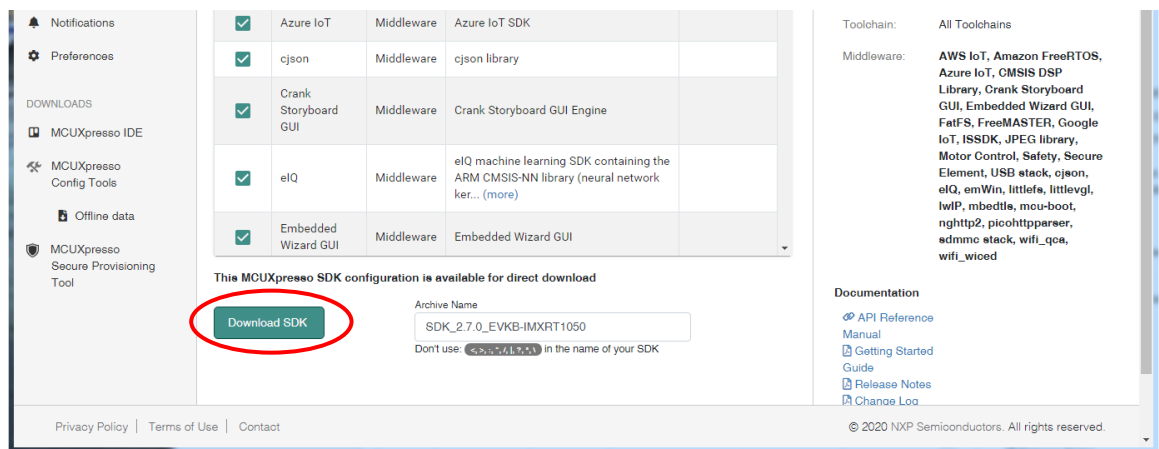


Figure 5 – MCUXpresso SDK Builder - SDK Builder

Also agree to the *Software Terms and Conditions* by clicking on the *I Agree* button.

Download of a (about) 170 MByte zip-file will begin.

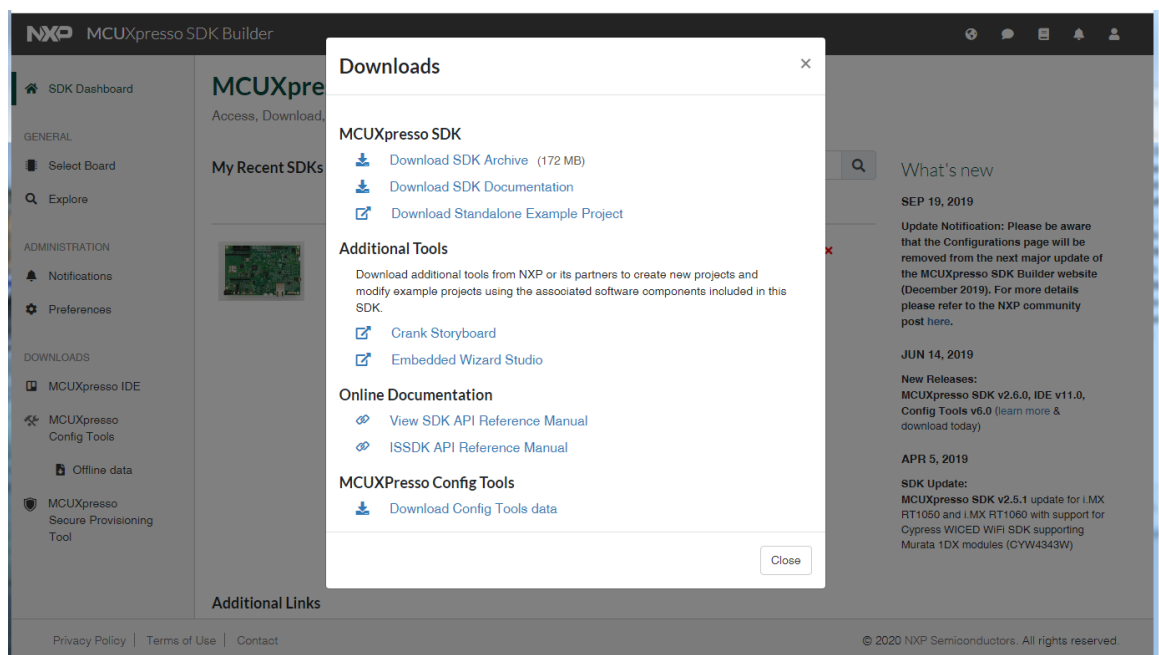


Figure 6 – MCUXpresso SDK Builder - SDK Downloads

Save the file on your computer. Note the download location. As of January 2020, the file name is **SDK_2.7.0_EVKB-IMXRT1050.zip** or **SDK_2.7.0_EVK-MIMXRT1060.zip** but the version number (i.e., 2.7.0) will increment over time. Extract the content of the zip-file to the desired installation location in your file system. Note that the installation root shall have a short path. For example: **c:\NXP_SDK**

2.2 Modify the SDK to Support iMX RT Developer's Kit

This section describes how to modify the previously downloaded SDK to be compatible with the features of the *iMX RT Developer's Kit*, like the EcoXiP.

The SDK supports hyper flash but the *iMX RT Developer's Kit* has EcoXiP flash from Adesto. Ideally the EcoXiP flash driver should be placed in a new file with `_ecoxip` in the name but that would require changes to every project for every IDE in the SDK and that is a huge task. The solution is instead to replace the file content but keep the filename. This way none of the projects have to be modified.

Download the zip-file from the *Resources* tab on the product page on Embedded Artists' website. The name of the file is:

imxrt1052_ea_files_<date>.zip, or

imxrt1062_ea_files_<date>.zip

Unpack the downloaded zip-file in a temporary directory, for example: `c:/temp/ea_files`

Copy

`c:/temp/ea_files/evkbimxrt1050_flexspi_nor_config.c`, or

`c:/temp/ea_files/evkmimxrt1060_flexspi_nor_config.c`

to

`<SDK_Install_Directory>/boards/evkbimxrt1050/xip/`, or

`<SDK_Install_Directory>/boards/evkmimxrt1060/xip/`

replacing the existing file. This will add support for the EcoXiP.

2.3 SDK Project Comments

The MCUXpresso SDK comes with a long list of example application code. To see what's available, browse to the boards folder of your SDK installation and select your board:

`<SDK_Install_Directory>/boards/evkbimxrt1050`, or

`<SDK_Install_Directory>/boards/evkmimxrt1060`

Only use the projects found under this sub-directory.

To learn more about specific example code, open the `readme.txt` file in an example's directory.

The following sections will describe some projects that will require extra configuration to work.

2.3.1 Common for All Examples

The zip archive with the patches, `imxrt1052_ea_files_<date>.zip` / `imxrt1062_ea_files_<date>.zip`, have a modified `clock_config.c` file that has already been modified so that the clock change described below is done automatically by reading fuses in the CPU to detect if it is Commercial or Industrial grade. The `clock_config.c` file also has the `VDD_SOC_IN` voltage set to 1.3V for stability.

For MCUXpresso IDE:

File to copy	Destination	Comment
<code>clock_config.c</code>	<code>board/</code>	Replace existing file

For Keil uVision or IAR Embedded Workbench:

File to copy	Destination	Comment
<code>clock_config.c</code>	Project root	Replace existing file

After copying the files, right-click the project and select *Refresh* to make the files appear in the project.

Why is the patch needed?

The iMX RT microprocessors come in two temperature grades: Commercial and Industrial. The Commercial grade has a 600MHz max core frequency and the Industrial has a 528MHz max core frequency. The NXP SDK is configured only for the Commercial grade microprocessors.

Clock	Commercial Grade "Over RUN mode"	Industrial Grade "Full RUN mode"
CPU	600 MHz	528 MHz
AHB	600 MHz	528 MHz
SEMC	150 MHz	132 MHz
IPG	150 MHz	132 MHz
PER	75 MHz	66 MHz
OSC	24 MHz	24 MHz
RTC	32768 Hz	32768 Hz

While it is possible to run the Industrial grade microprocessor in the Over RUN mode (i.e. 600 MHz), doing so for extended periods of time will reduce the lifespan so it is strongly recommended to modify the software to run in Full RUN mode (i.e. 528 MHz)

There are two examples called **power_mode_switch** and **power_mode_switch_bm** in the SDK that demonstrates different power modes and how they affect the different clocks. The clock settings are also described in NXP's User Manual for each of the microprocessors.

To make sure that the SDK examples use the correct frequency and

To change the SDK examples to the 528 MHz speed on an Industrial board, open `clock_config.c` and replace this line

```
CLOCK_SetMux(kCLOCK_PrePeriphMux, 3);
```

with

```
CLOCK_SetMux(kCLOCK_PrePeriphMux, 0);
```

At the end of the same function in `clock_config.c` replace this line

```
SystemCoreClock = BOARD_BOOTCLOCKRUN_CORE_CLOCK;
```

with the following statement to correctly update the `SystemCoreClock` variable

```
SystemCoreClockUpdate();
```

2.3.2 USB Examples

The USB interface is different on the NXP EVK that the SDK is for and the *iMX RT Developer's Kit*. The USB examples all have a block of code (most of the time in app.h) to define the CONTROLLER_ID like this:

```
#if ((defined USB_HOST_CONFIG_KHCI) && (USB_HOST_CONFIG_KHCI))
#define CONTROLLER_ID kUSB_ControllerKhci0
#endif /* USB_HOST_CONFIG_KHCI */
#if ((defined USB_HOST_CONFIG_EHCI) && (USB_HOST_CONFIG_EHCI))
#define CONTROLLER_ID kUSB_ControllerEhci0
#endif /* USB_HOST_CONFIG_EHCI */
#if ((defined USB_HOST_CONFIG_OHCI) && (USB_HOST_CONFIG_OHCI))
#define CONTROLLER_ID kUSB_ControllerOhci0
#endif /* USB_HOST_CONFIG_OHCI */
#if ((defined USB_HOST_CONFIG_IP3516HS) && (USB_HOST_CONFIG_IP3516HS))
#define CONTROLLER_ID kUSB_ControllerIp3516Hs0
#endif /* USB_HOST_CONFIG_IP3516HS */
```

If the USB example is not working, then try to change the 0 (marked in red above) to a 1 and test again. Note that in version 2.5.0 of the SDK several of the examples work without the modification as the code is written to enable both USB interfaces.

2.3.3 Network Examples

The *iMX RT1052 OEM* and *iMX RT1062 OEM* board have an EEPROM with a unique id that can be used as MAC address in the network examples, found in the demo_apps/lwip directory. Follow these steps to use it:

1. Start by copying the mac_addr.c and mac_addr.h files from the imxrt1052_ea_files_<date>.zip / imxrt1062_ea_files_<date>.zip archive in section 2.2 into your project
2. Add the mac_addr.c file to the project. How this is done depends on the IDE but for Keil uVision, right click on the source folder in the project and select *Add existing files to group 'source'*.
3. Add the I2C driver to the project the same way as in the previous step. The driver is called fsl_lpi2c and can be found in the <SDK_Install_Directory>/devices/MIMXRT1052/drivers/ folder or <SDK_Install_Directory>/devices/MIMXRT1062/drivers/ folder.
4. Add #include "mac_addr.h" to main.c after the other include statements
5. Find the declaration in the main.c file that looks like this:

```
ethernetif_config_t fsl_enet_config0 = {
    .phyAddress = EXAMPLE_PHY_ADDRESS,
    .clockName = EXAMPLE_CLOCK_NAME,
    .macAddress = configMAC_ADDR,
};
```

and add this line after it:

```
MAC_Read(fsl_enet_config0.macAddress);
```

Now the program will attempt to read the MAC address from the EEPROM before the LWIP stack is initialized.

2.3.4 Wi-Fi Examples

The *iMX RT Developer's Kits* support M.2 modules. As of January 2020 SDK 2.7.0 contained two projects that support this module: *wiced_iperf* and *wiced_mfg_test*. The SDK for the *iMX RT1052 OEM* board has an additional example for iperf3: *wiced_iperf3_4343W*. The hardware setup is a bit different for the *iMX RT*

Developer's Kit and a number of files must be copied into the project from the `imxrt1052_ea_files_<date>.zip` / `imxrt1062_ea_files_<date>.zip` archive in section 2.2

For MCUXpresso IDE:

File to copy	Destination	Comment
<code>pin_mux.c</code>	<code>board/</code>	Replace existing file
<code>fsl_lpi2c.c</code>	<code>drivers/</code>	New file
<code>fsl_lpi2c.h</code>	<code>drivers/</code>	New file
<code>pca6416.c</code>	<code>source/</code>	New file
<code>pca6416.h</code>	<code>source/</code>	New file
<code>wwd_platform.c</code>	<code>wiced/43xxx_Wi-Fi/WICED/platform/MCU/LPC/WWD/</code>	Replace existing file
<code>wwd_SDIO.c</code>	<code>wiced/43xxx_Wi-Fi/WICED/platform/MCU/LPC/WWD/</code>	Replace existing file

After copying the files, right-click the project and select *Refresh* to make the files appear in the project.

For Keil uVision or IAR Embedded Workbench:

File to copy	Destination	Comment
<code>pin_mux.c</code>	Project root	Replace existing file
<code>fsl_lpi2c.c</code>	Project root	New file
<code>fsl_lpi2c.h</code>	Project root	New file
<code>pca6416.c</code>	Project root	New file
<code>pca6416.h</code>	Project root	New file
<code>wwd_platform.c</code>	<code>middleware/wiced/43xxx_Wi-Fi/WICED/platform/MCU/LPC/WWD</code>	Replace existing file
<code>wwd_SDIO.c</code>	<code>middleware/wiced/43xxx_Wi-Fi/WICED/platform/MCU/LPC/WWD</code>	Replace existing file

After copying the files into the project the IDE must be told to look for the new files. How this is done depends on the IDE but for Keil uVision, right click on for example the *source* folder in the project and select *Add existing files to group 'source'*.

2.3.5 Bluetooth Low Energy (BLE) Example

The *iMX RT Developer's Kits* support M.2 modules. As of January 2020 SDK 2.7.0 contained one project that supports BLE for the 1DX M.2 module: *wiced_ble_4343W*.

The hardware setup is a bit different for the *iMX RT Developer's Kit* and a number of files must be copied into the project from the `imxrt1052_ea_files_<date>.zip` / `imxrt1062_ea_files_<date>.zip` archive in section 2.2

For MCUXpresso IDE:

File to copy	Destination	Comment
<code>pin_mux_ble.c</code>	<code>board/pin_mux.c</code>	Replace existing file

fsl_lpi2c.c	drivers/	New file
fsl_lpi2c.h	drivers/	New file
pca6416.c	source/	New file
pca6416.h	source/	New file
wwd_platform.c	wiced/43xxx_Wi-Fi/WICED/platform/MCU/LPC/WWD/	Replace existing file
wwd_SDIO.c	wiced/43xxx_Wi-Fi/WICED/platform/MCU/LPC/WWD/	Replace existing file
platform_imxrt.c	wiced/43xxx_BLE/wiced_bt/imxrt_port/	Replace existing file

After copying the files, right-click the project and select *Refresh* to make the files appear in the project.

For Keil uVision or IAR Embedded Workbench:

File to copy	Destination	Comment
pin_mux_ble.c	Project root. Rename to pin_mux.c.	Replace existing file
fsl_lpi2c.c	Project root	New file
fsl_lpi2c.h	Project root	New file
pca6416.c	Project root	New file
pca6416.h	Project root	New file
wwd_platform.c	middleware/wiced/43xxx_Wi-Fi/WICED/platform/MCU/LPC/WWD/	Replace existing file
wwd_SDIO.c	middleware/wiced/43xxx_Wi-Fi/WICED/platform/MCU/LPC/WWD/	Replace existing file
platform_imxrt.c	middleware/wiced/43xxx_BLE/wiced_bt/imxrt_port/	Replace existing file

After copying the files into the project the IDE must be told to look for the new files. How this is done depends on the IDE but for Keil uVision, right click on for example the *source* folder in the project and select *Add existing files to group 'source'*.

2.3.6 Bluetooth Passthrough Example

The *iMX RT Developer's Kits* support M.2 modules. As of January 2020 SDK 2.7.0 contained one project that support Bluetooth passthrough (it allows the PC to talk directly to the M.2 module over a UART connection) for the 1DX M.2 module: *wiced_bt_passthrough_4343W*.

Note that for this program to work an FTDI TTL-232R-3V3 cable must be connected to the J36 connector on the *iMX RT Developer's Kit* and it is that COM port that should be used in CyBlueTool.

The hardware setup is a bit different for the *iMX RT Developer's Kit* and a number of files must be copied into the project from the *imxrt1052_ea_files_<date>.zip* / *imxrt1062_ea_files_<date>.zip* archive in section 2.2

For MCUXpresso IDE:

File to copy	Destination	Comment
--------------	-------------	---------

pin_mux_bt_passthrough.c	board/pin_mux.c	Replace existing file
fsl_lpi2c.c	drivers/	New file
fsl_lpi2c.h	drivers/	New file
pca6416.c	source/	New file
pca6416.h	source/	New file
wiced_bt_passthrough.c	source/	Replace existing file

After copying the files, right-click the project and select *Refresh* to make the files appear in the project.

For Keil uVision or IAR Embedded Workbench:

File to copy	Destination	Comment
pin_mux_bt_passthrough.c	Project root. Rename to pin_mux.c.	Replace existing file
fsl_lpi2c.c	Project root	New file
fsl_lpi2c.h	Project root	New file
pca6416.c	Project root	New file
pca6416.h	Project root	New file
wiced_bt_passthrough.c	Project root	Replace existing file

After copying the files into the project the IDE must be told to look for the new files. How this is done depends on the IDE but for Keil uVision, right click on for example the *source* folder in the project and select *Add existing files to group 'source'*.

2.3.7 Crank Storyboard CoffeeApp Example

As of January 2020 SDK 2.7.0 contained two demo projects from Crank Software: *crank_storyboard_coffeeapp* and *crank_storyboard_helloworld*. The HelloWorld program works as expected but the CoffeeApp program will not fit in the flash available on the *iMX RT Developer's Kits*.

2.3.8 Examples using SD Card (only applicable to iMX RT1062 Developer's Kit)

The pinning for the SD Card interface is incorrect for the iMX RT1062 Developer's Kit. The reset GPIO should be IOMUXC_GPIO_AD_B0_05_GPIO1_IO05 and not IOMUXC_GPIO_AD_B1_03_GPIO1_IO19.

Affected projects:

- sdmmc_examples\sdcard_fatfs\
- sdmmc_examples\sdcard_fatfs_freertos\
- sdmmc_examples\sdcard_freertos\
- sdmmc_examples\sdcard_interrupt\
- sdmmc_examples\sdcard_polling\
- usb_examples\usb_device_composite_cdc_msc_sdcard\
- usb_examples\usb_device_composite_cdc_msc_sdcard_lite\

- usb_examples\usb_device_msc_sdcard\
- usb_examples\usb_device_msc_sdcard_lite\

Correcting the pinning must be done in two files:

1. Open board.h and replace

```
#define BOARD_SD_POWER_RESET_GPIO_PIN (19U) with  
#define BOARD_SD_POWER_RESET_GPIO_PIN (5U)
```

2. Open pin_mux.c and replace

```
IOMUXC_GPIO_AD_B1_03_GPIO1_IO19 with  
IOMUXC_GPIO_AD_B0_05_GPIO1_IO05
```

This define is used both in call to `IOMUXC_SetPinMux()` and in the call to `IOMUXC_SetPinConfig()` so make sure to change both.

2.4 Getting Started With a Specific IDE

The following chapters will describe how to get started with a number of different IDEs.

Chapter 3 describes Keil uVision and the SDK.

Chapter 4 describes Keil uVision and RTE.

Chapter 5 describes IAR Embedded Workbench.

Chapter 6 describes NXP MCUXpresso.

3 Getting Started with Keil uVision/MDK and SDK

This section is a guide to open, build and debug an example application using Keil uVision/MDK with the SDK as installed in chapter 2 . It is assumed that you have this development environment installed on your computer.

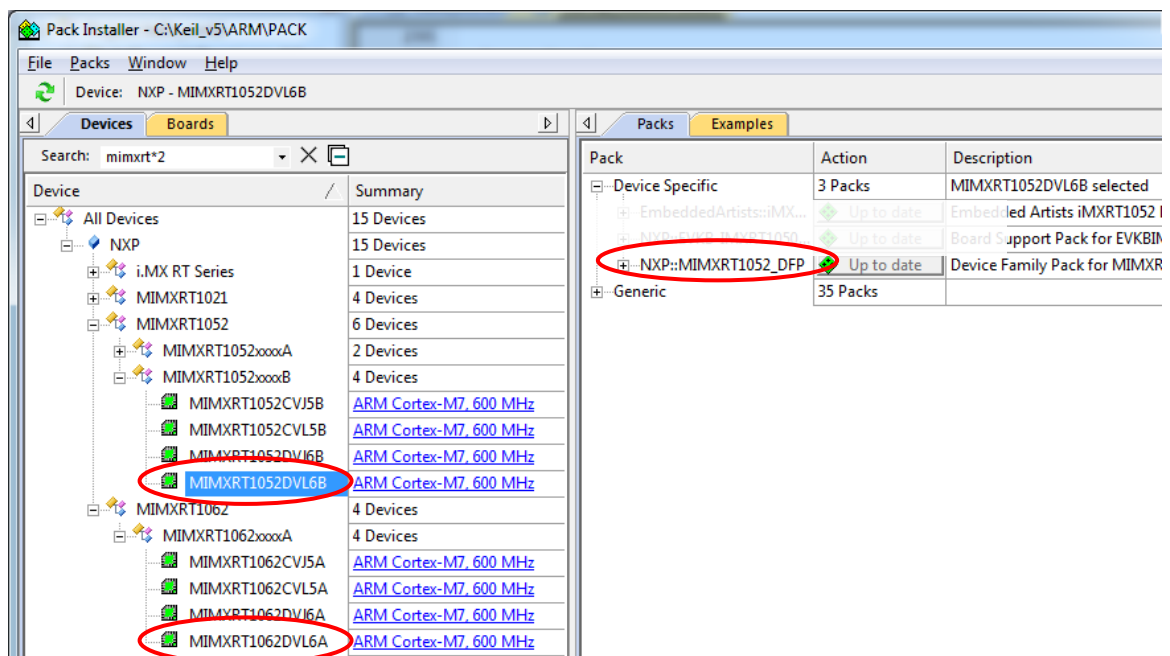
Following the instructions in this section will make it possible to run all examples in the downloaded SDK. Note that none of the examples use the RTE (Run-Time Environment) feature of the Keil uVision/MDK. The use of RTE is described in chapter 4 .

Make sure the version of uVision is v5.25.2, or later. Otherwise there is no support for flash download/programming to the EcoXiP flash memory directly.

3.1 Install CMSIS Device Pack

After the MDK tools are installed, Cortex Microcontroller Software Interface Standard (CMSIS) device packs must be installed to fully support the device from a debug perspective. These packs include things such as memory map information, register definitions and flash programming algorithms. Follow these steps to install the IMXRT105x CMSIS pack or IMXRT106x CMSIS pack.

1. Start Keil uVision
2. Start the Pack Installer tool with this button on the toolbar: 
3. Browse to the MIMXRT1052 in the device tree (NXP→MIMXRT1052→MIMXRT1052xxxxB) or MIMXRT1062 in the device tree (NXP→MIMXRT1062→MIMXRT1062xxxxA) and then click the Install button next to the package with a "_DFP" in the name. Note that the button in the image below has "Up to date" as the package has already been installed.



3.2 Build an Example Application

The following steps will guide you through opening the hello_world application. These steps may change slightly for other example applications as some of these applications may have additional layers of folders in their path.

1. If not already done, open the desired example application workspace in:
`<install_dir>/boards/evkbimxrt1052/<example_type>/<application_name>/mdk`
 The workspace file is named `<demo_name>.uvmpw`, so for this specific example, the actual path is:
`<install_dir>/boards/evkbimxrt1052/demo_apps/hello_world/mdk/hello_world.uvmpw`

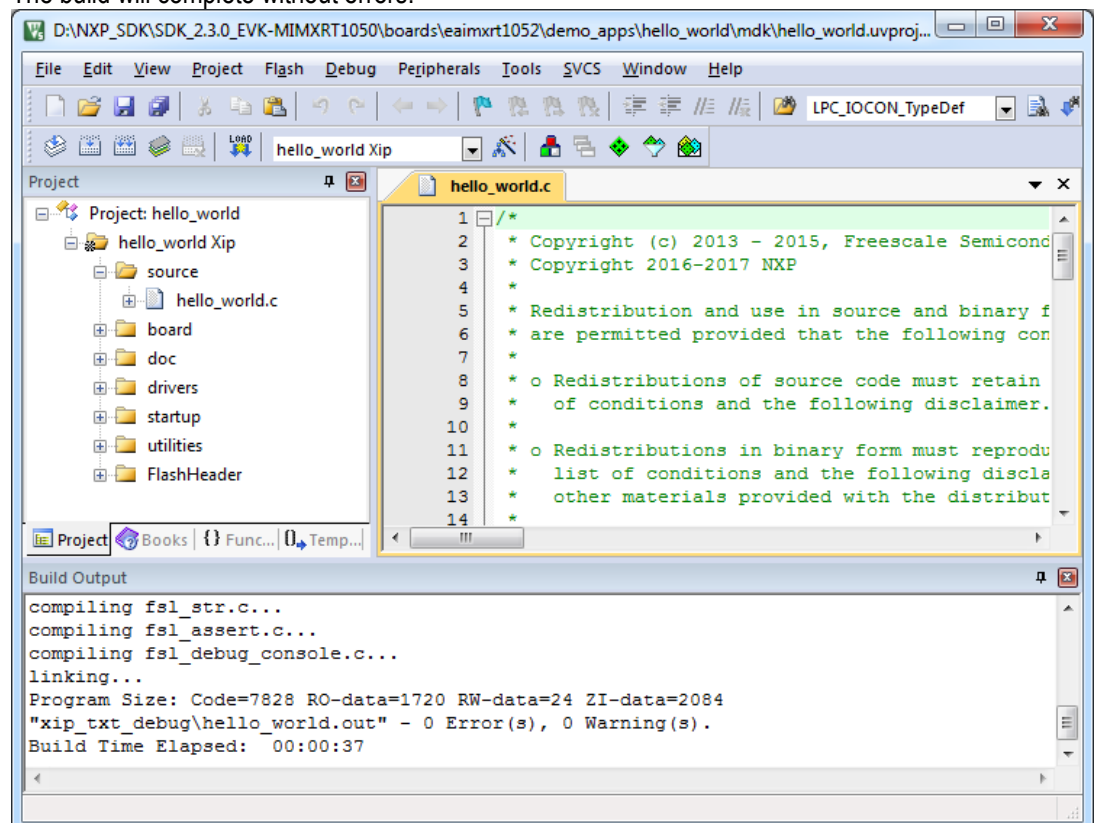
or

- `<install_dir>/boards/evkmimxrt1062/<example_type>/<application_name>/mdk`
 The workspace file is named `<demo_name>.uvmpw`, so for this specific example, the actual path is:
`<install_dir>/boards/evkmimxrt1062/demo_apps/hello_world/mdk/hello_world.uvmpw`

2. To build the demo project, select the "Rebuild" button, highlighted in red.



3. The build will complete without errors.

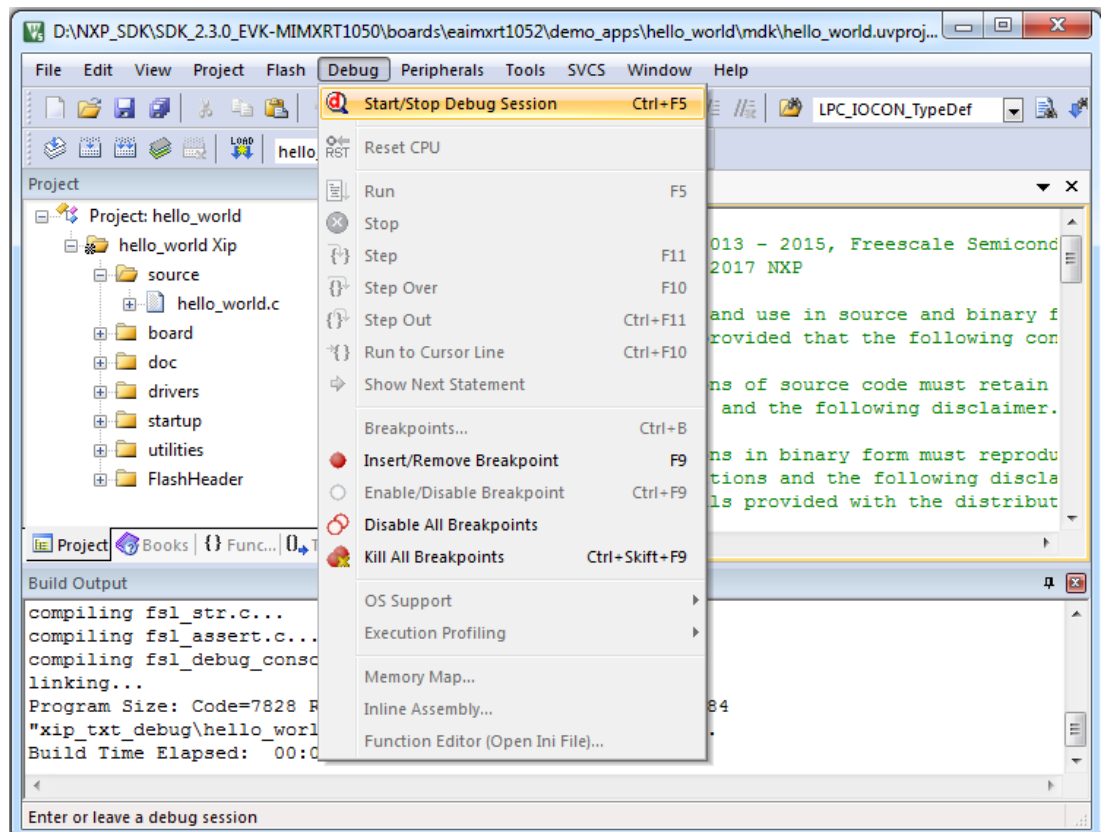


3.3 Run an Example Application

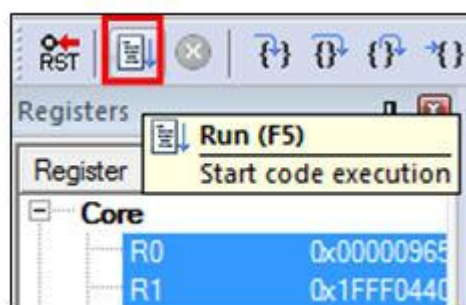
To download and run the application, perform these steps:

4. Connect the debug probe to development platform to your PC via USB cable. See chapter 7 for details how to connect the debug probe.

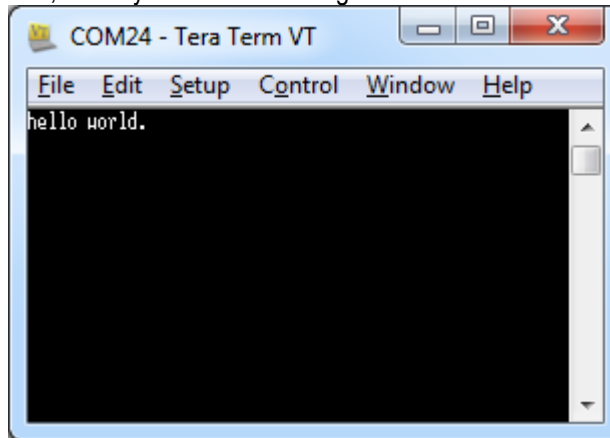
5. Select the *Debug* menu and then *Start/Stop Debug Session*, or simply press Ctrl+F5. The application will then be downloaded into SRAM.



6. Open the terminal application on the PC, such as TeraTerm or PuTTY, and connect to the debug serial port number. Configure the terminal with 115200 baud, 8N1. You can alter the baud rate by searching for the reference `BOARD_DEBUG_UART_BAUDRATE` variable in file: **board.h**
7. Run the code by clicking the "Run" button (or press F5) to start the application.



8. The `hello_world` application is now running and a banner is displayed on the terminal. If this is not true, check your terminal settings and connections.



3.4 Selecting Build Target

A target is a variant of the project that for example can change compiler flags or dictate in what memory region the program should execute. All projects have one or more targets as shown in the picture below.

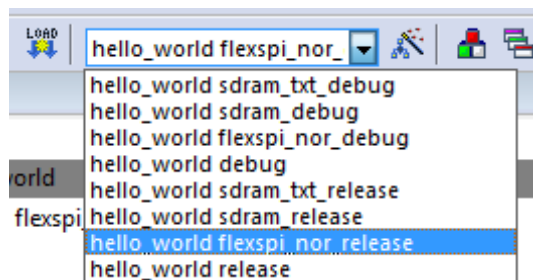


Figure 7 – Build Targets in Keil uVision/MDK

The target name starts with the project name, **hello_world** in this case, and then the nomenclature as outlined in the table below is used. Release targets have a higher compiler optimization degree making the target less suitable for debugging.

<project> sdram_debug <project> sdram_release	The application runs in internal SRAM but with data in external SDRAM
<project> sdram_txt_debug <project> sdram_txt_release	The application runs in external SDRAM but with data in internal SRAM
<project> debug <project> release	The application runs in internal SRAM
<project> flexspi_nor_debug <project> flexspi_nor_release	The application runs in external flash, which must be programmed/flushed before use. Need extra configuration, see section 3.5 .

Note that each of the targets have its own set of options so changing for example include path in one target does not affect the other targets.

3.5 Configuring Flash Algorithm

To setup a project for storing/executing the application from the EcoXiP flash, select one of the `flexspi_nor_*` targets, right click on the project and select *Options for Target <name>* (or press Alt+F7). Select the *Debug* tab and select *Settings* for the debugger. Then select the *Flash Download* tab and replace the existing driver with the EcoXiP flash driver as shown below.

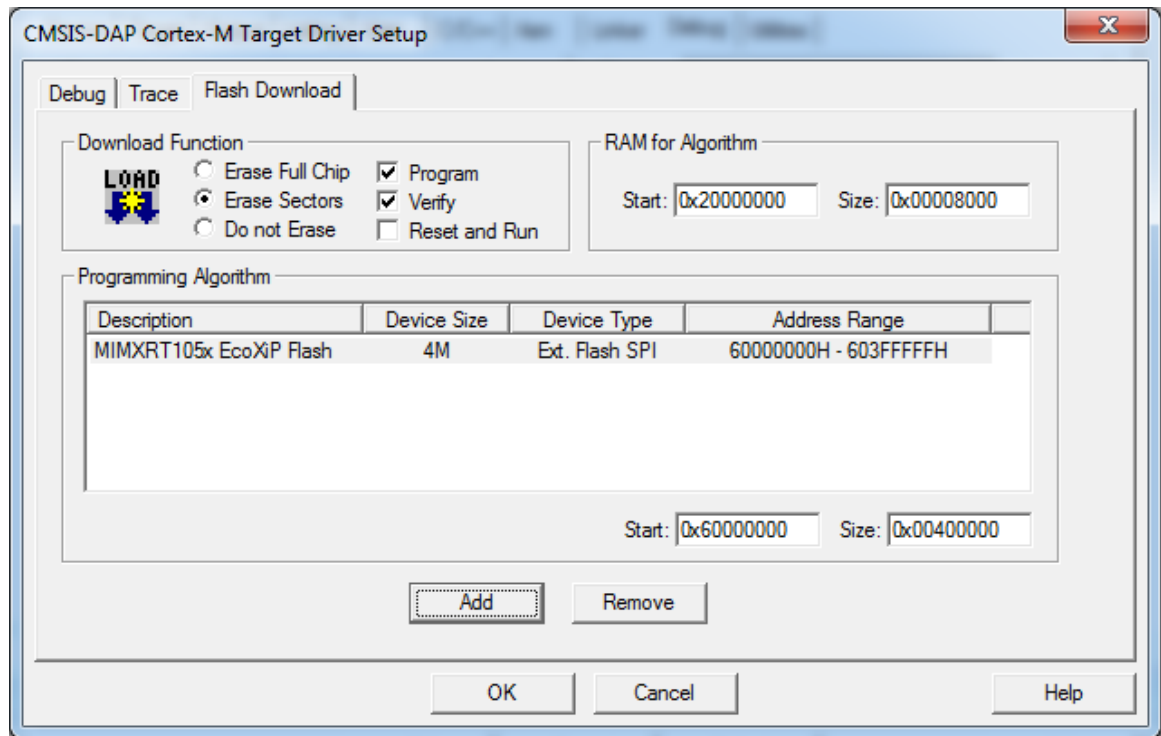


Figure 8 – Keil uVision/MDK Flash Algorithm Settings

4 Getting Started with Keil uVision/MDK and RTE

This section is a guide to open, build and debug an example application using Keil uVision/MDK with the Board Support Package (BSP) installed using the Pack Installer and not the SDK described in chapter 2 . It is assumed that you have this development environment installed on your computer.

The BSP contains a small subset of all the examples available in the SDK. The main difference between the SDK and the BSP examples is that the BSP examples use the Run-Time Environment (RTE) feature in Keil uVision/MDK and the SDK doesn't.

The RTE is a way to manage software components (e.g. drivers, rtos, network stacks, usb stacks) to add/remove features in a project without having to add each source file and include path individually.

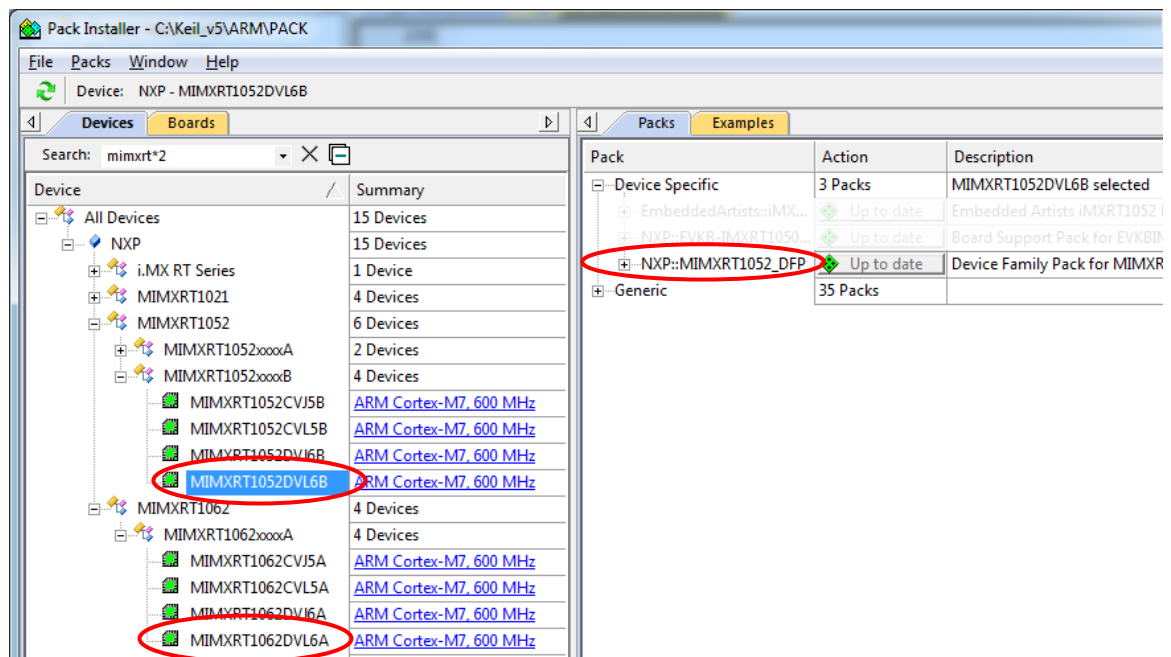
The recommendation is to use the SDK unless you know exactly what you are doing. The use of the SDK is described in chapter 3 .

Make sure the version of uVision is v5.25.2, or later. Else there is no support for flash download/programming to the EcoXiP flash memory directly.

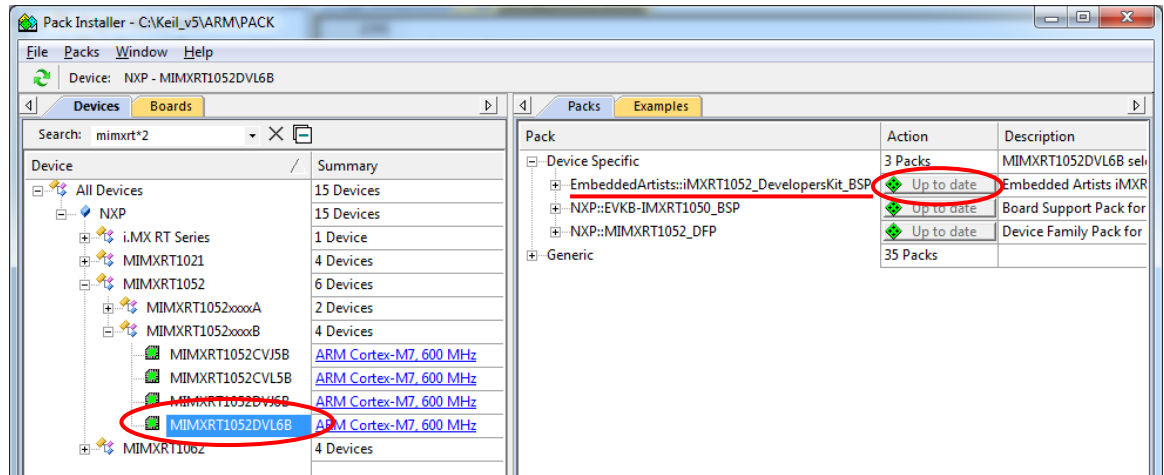
4.1 Install CMSIS Device Pack

After the MDK tools are installed, Cortex Microcontroller Software Interface Standard (CMSIS) device packs must be installed to fully support the device from a debug perspective. These packs include things such as memory map information, register definitions and flash programming algorithms. Follow these steps to install the IMXRT105x CMSIS pack or IMXRT106x CMSIS pack.

1. Start Keil uVision
2. Start the Pack Installer tool with this button on the toolbar: 
3. Browse to the MIMXRT1052 in the device tree (NXP→MIMXRT1052→MIMXRT1052xxxxB) or MIMXRT1062 in the device tree (NXP→MIMXRT1062→MIMXRT1062xxxx) and then click the Install button next to the package name. Note that the button in the image below has "Up to date" as the package has already been installed.



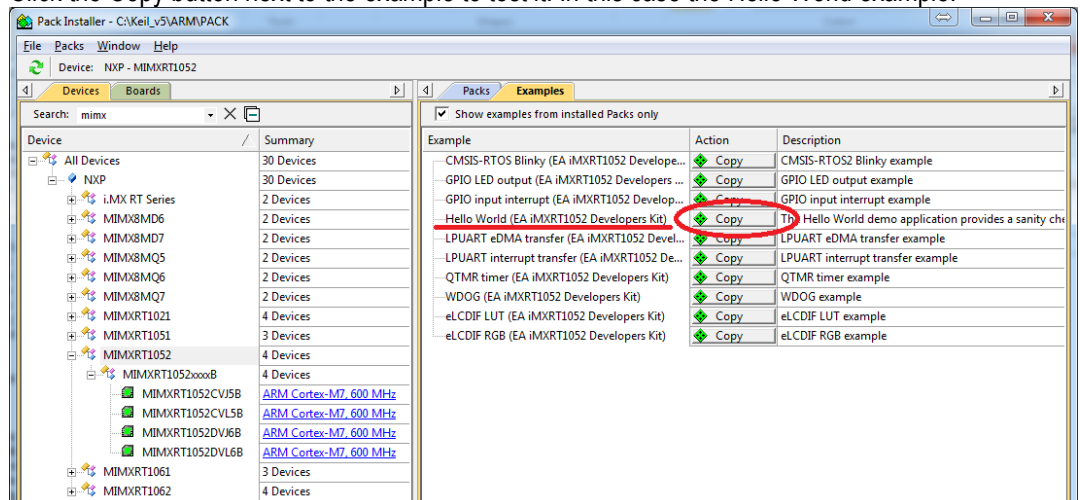
- Next click the Install button for the EmbeddedArtists::iMXRT1052_DevelopersKit_BSP or EmbeddedArtists::iMXRT1062_DevelopersKit_BSP. Note that the button in the image below has "Up to date" as the package has already been installed



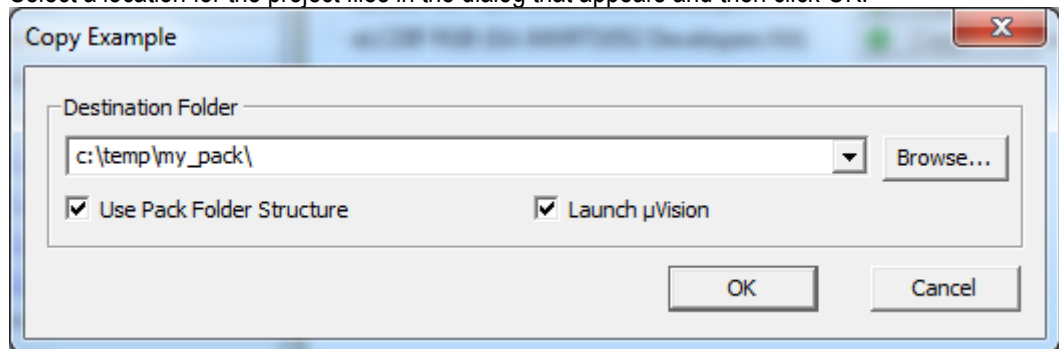
4.2 Build an Example Application

The following steps will guide you through opening the Hello World application. These steps may change slightly for other example applications as some of these applications may have additional layers of folders in their path.

- Start Keil uVision
- Start the Pack Installer tool with this button on the toolbar: 
- Browse to the MIMXRT1052 in the device tree (NXP→MIMXRT1052→MIMXRT1052xxxxB) or MIMXRT1062 in the device tree (NXP→MIMXRT1062→MIMXRT1062xxxxB) and then select the Examples tab
- Click the Copy button next to the example to test it. In this case the Hello World example.



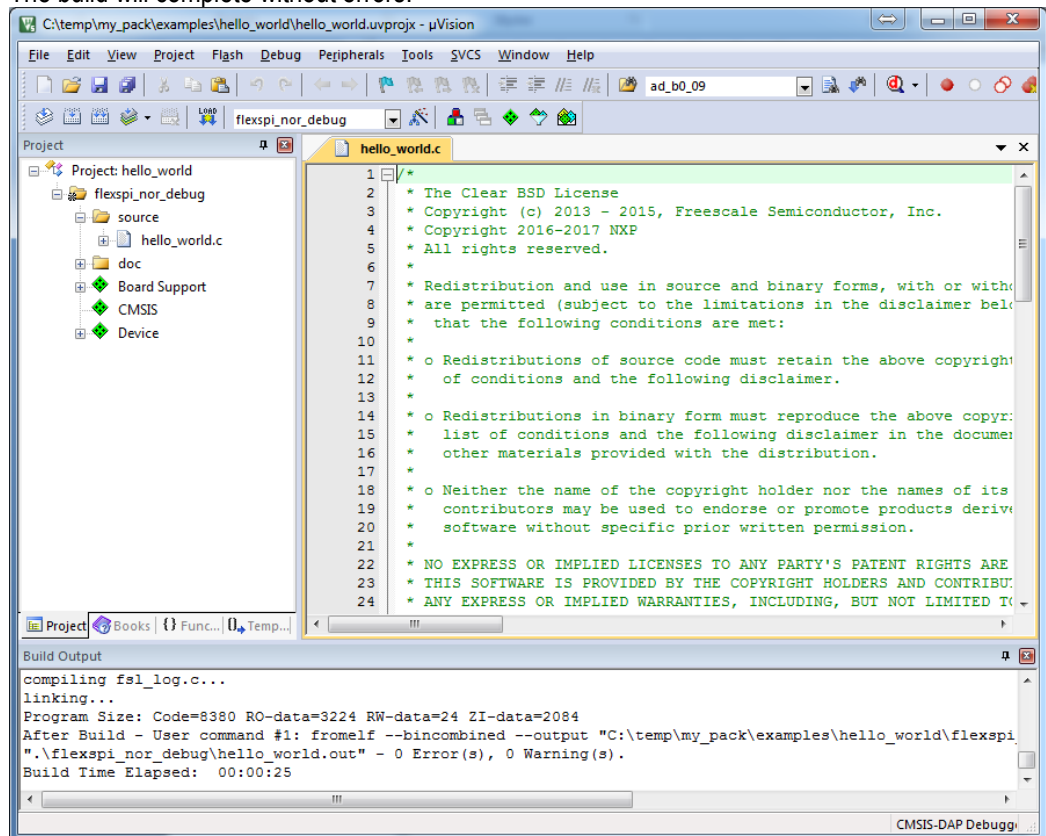
5. Select a location for the project files in the dialog that appears and then click OK.



6. A new uVision window will appear after a few seconds with the newly created project
7. To build the demo project, select the "Rebuild" button, highlighted in red.



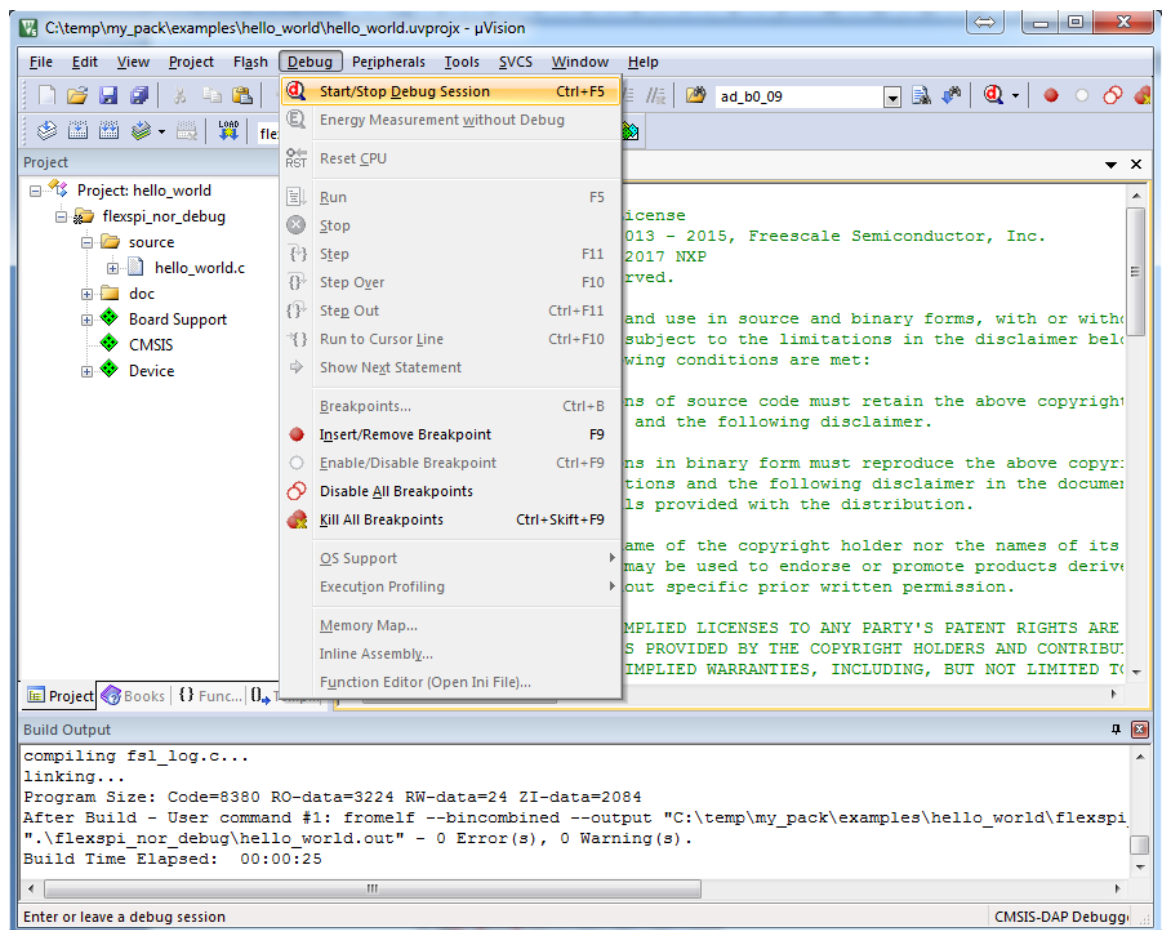
8. The build will complete without errors.



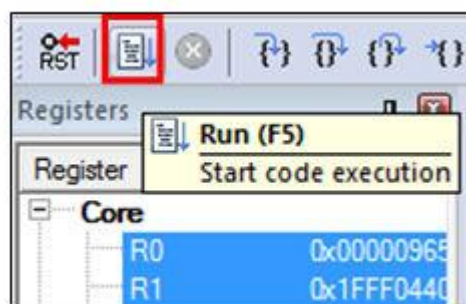
4.3 Run an Example Application

To download and run the application, perform these steps:

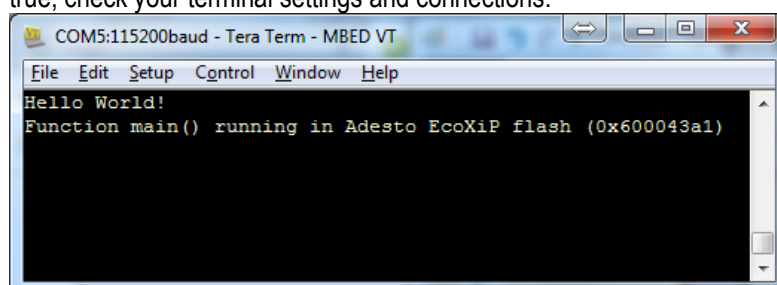
1. Connect the debug probe to development platform to your PC via USB cable. See chapter 7 for details how to connect the debug probe.
2. Select the *Debug* menu and then *Start/Stop Debug Session*, or simply press Ctrl+F5. The application will then be downloaded into flash and started.



3. Open the terminal application on the PC, such as TeraTerm or PuTTY, and connect to the debug serial port number. Configure the terminal with 115200 baud, 8N1. You can alter the baud rate by searching for the reference BOARD_DEBUG_UART_BAUDRATE variable in file: **board.h**
4. Run the code by clicking the “Run” button (or press F5) to start the application.



5. The Hello World application is now running and a banner is displayed on the terminal. If this is not true, check your terminal settings and connections.



4.4 Selecting Build Target

A target is a variant of the project that for example can change compiler flags or dictate in what memory region the program should execute. All projects have one or more targets as shown in the picture below.

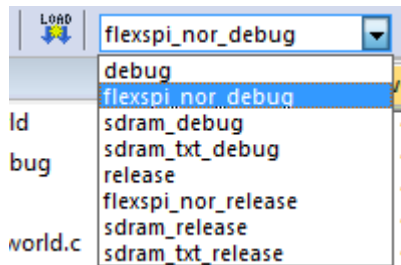


Figure 9 – Build Targets in Keil uVision/MDK

The default target is **flexspi_nor_debug**. The table below shows each of the targets. Release targets have a higher compiler optimization degree making the target less suitable for debugging.

sdram_debug sdram_release	The application runs in internal SRAM but with data in external SDRAM
sdram_txt_debug sdram_txt_release	The application runs in external SDRAM but with data in internal SRAM
debug release	The application runs in internal SRAM
flexspi_nor_debug flexspi_nor_release	The application runs in external flash, which must be programmed/flashed before use. Need extra configuration, see section 4.5 .

Note that each of the targets have its own set of options so changing for example include path in one target does not affect the other targets.

4.5 Configuring Flash Algorithm

To setup a project for storing/executing the application from the EcoXiP flash, select one of the `flexspi_nor_*` targets, right click on the project and select *Options for Target <name>* (or press Alt+F7). Select the *Debug* tab and select *Settings* for the debugger. Then select the *Flash Download* tab. Replace the existing driver with the EcoXiP flash driver as shown below.

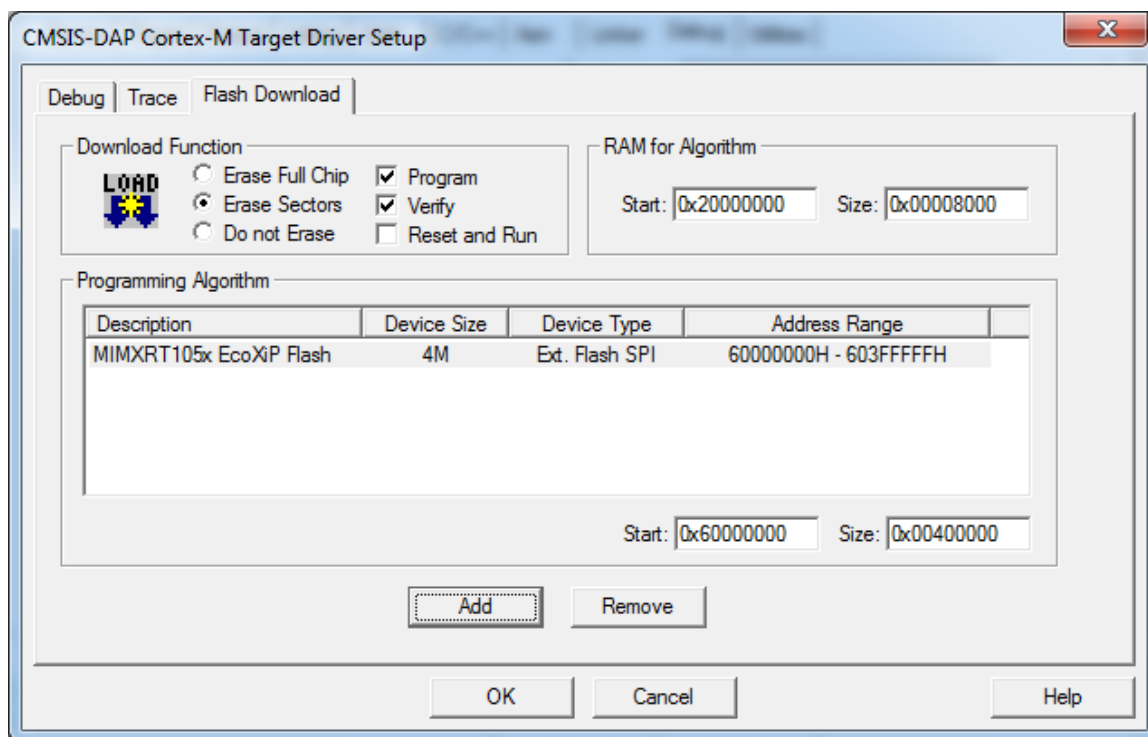
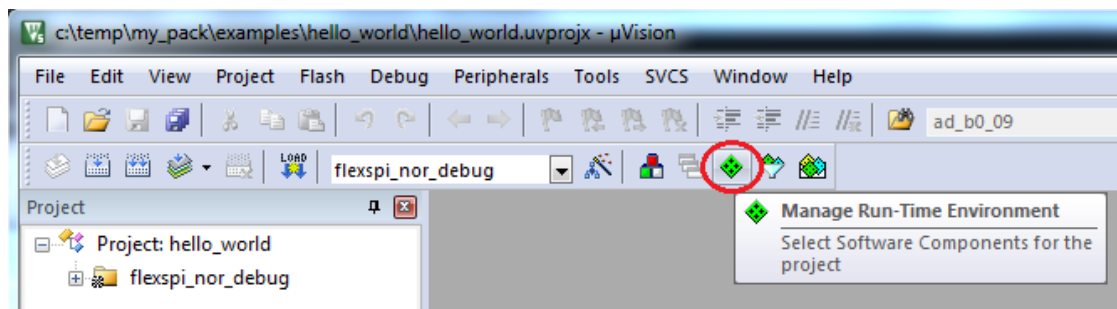


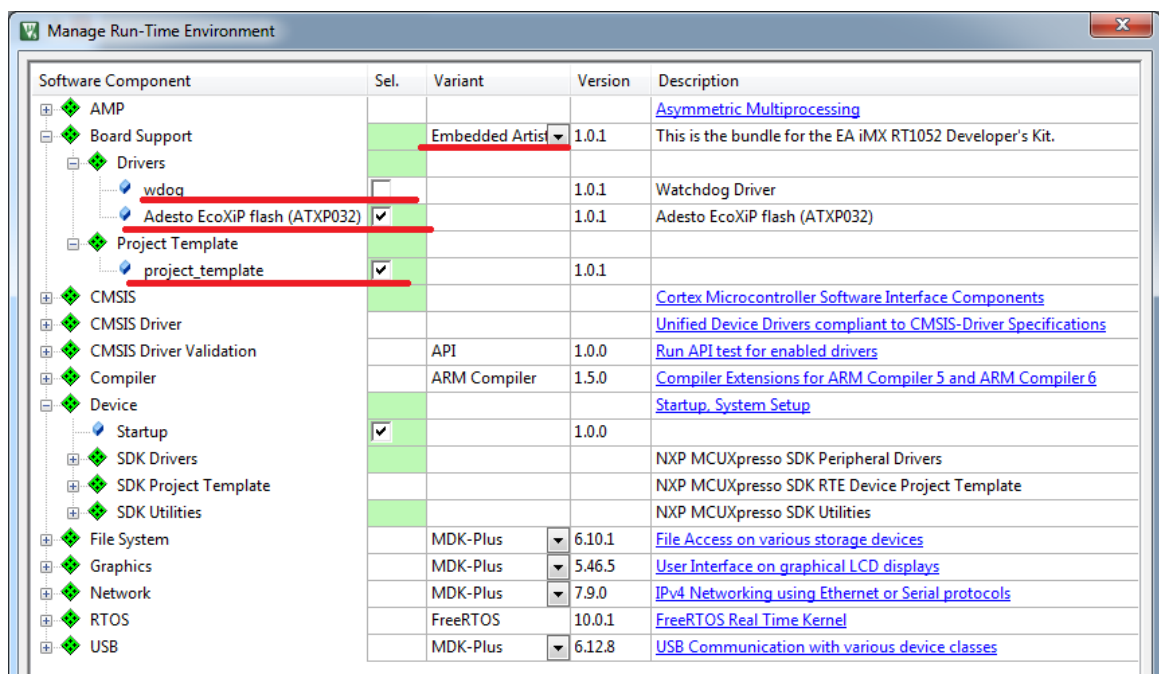
Figure 10 – Keil uVision/MDK Flash Algorithm Settings

4.6 Working with RTE

The Manage Run-Time Environment tool is started with a button on the toolbar:



The example projects are already configured correctly and it is possible to add more software components by ticking the corresponding checkboxes.



There are a couple of very important things to note:

- 1) The Board Support must be "Embedded Artists" to get the drivers and templates as seen above
- 2) The Adesto EcoXip flash (ATXP032) must be selected otherwise applications will not start after being flashed
- 3) If the watchdog driver (wdog) is used it must be selected from Board Support->Drivers->wdog and not the Device->SDK Drivers->wdog. The reason is that the one in Board Support is patched to work with the hardware.

The system for managing the RTE is described by Keil here:

http://www.keil.com/support/man/docs/uv4/uv4_ca_rtemanager.htm .

5 Getting Started with IAR Embedded Workbench

This section is a guide to open, build and debug an example application using IAR Embedded Workbench. It is assumed that you have this development environment installed on your computer.

Make sure the version of Embedded Workbench is 8.30.1, or later. Else there is no support for flash download/programming to the EcoXiP flash memory directly.

Every project that use the flash must be manually modified as they come pre-configured with the wrong flash type. The instructions for this can be found in section 5.3 .

Many of the SRAM/SDRAM sample projects created for NXP's EVK will run on the iMX RT Developer's Kit, but not all of them.

5.1 Build an Example Application

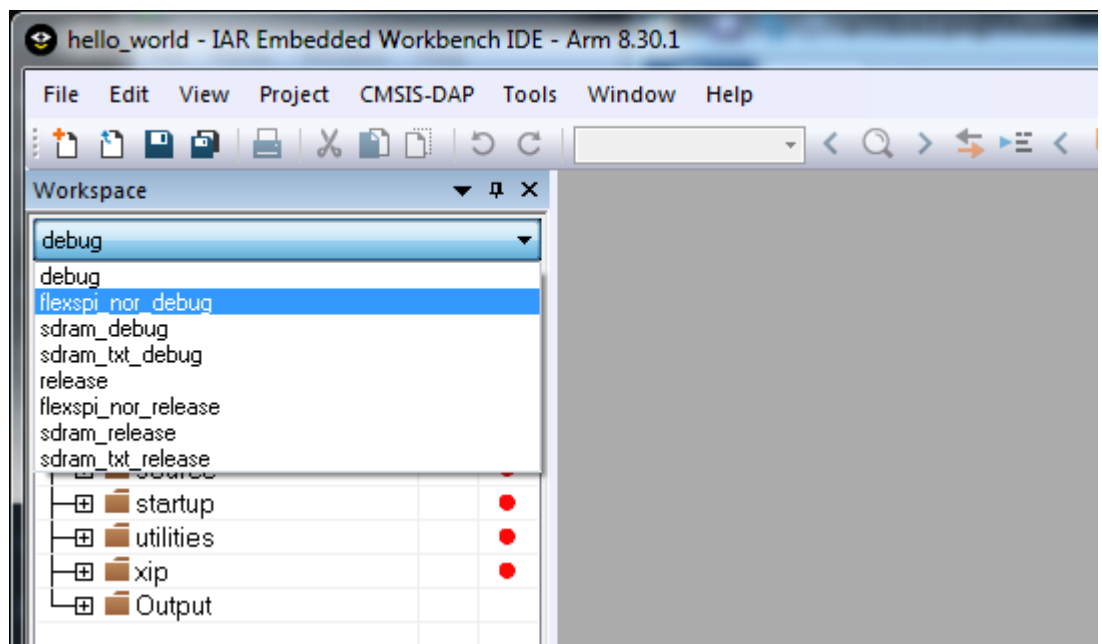
The following steps will guide you through opening the hello_world application. These steps may change slightly for other example applications as some of these applications may have additional layers of folders in their path.

1. If not already done, open the desired example application workspace in:
`<install_dir>/boards/evkbimxrt1050/<example_type>/<application_name>/iar`
The workspace file is named `<demo_name>.eww`, so for this specific example, the actual path is:
`<install_dir>/boards/evkbimxrt1050/demo_apps/hello_world/iar/hello_world.eww`

or

`<install_dir>/boards/evkmimxrt1060/<example_type>/<application_name>/iar`
The workspace file is named `<demo_name>.eww`, so for this specific example, the actual path is:
`<install_dir>/boards/evkmimxrt1060/demo_apps/hello_world/iar/hello_world.eww`

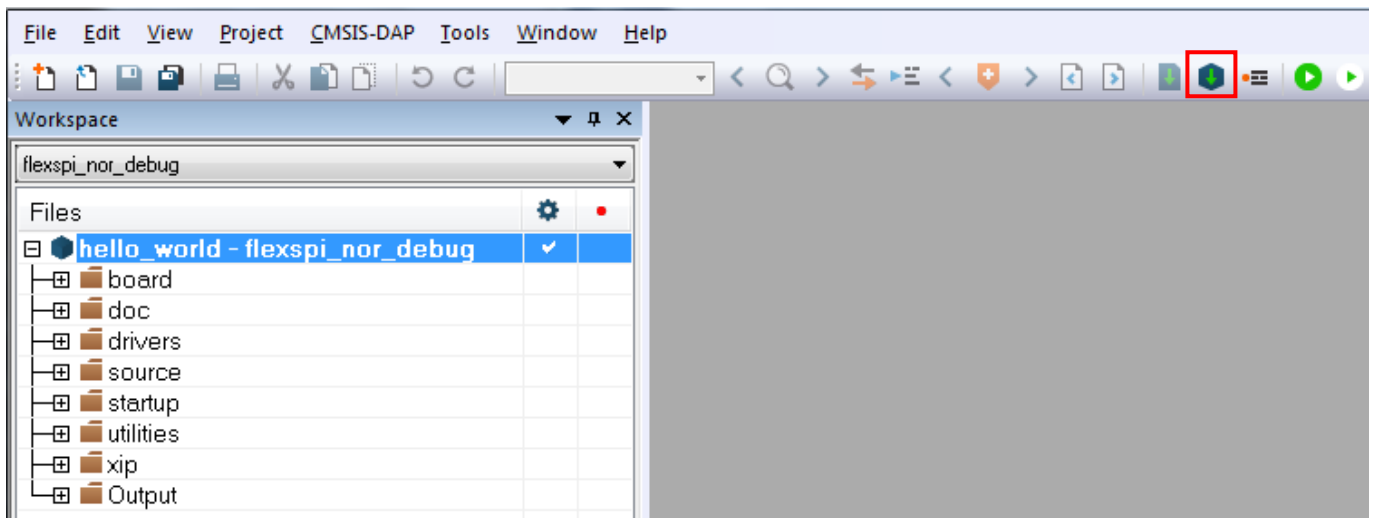
2. Select the desired build target from the drop-down.
For this example, select the "flexspi_nor_debug" target.



The target name indicates from where the application is executing, see table below. The name also indicates if it is a release or debug target. Release targets have a higher compiler optimization degree making the target less suitable for debugging.

sdram_debug sdram_release	The application runs in internal SRAM but with data in external SDRAM
sdram_txt_debug sdram_txt_release	The application runs in external SDRAM but with data in internal SRAM
debug release	The application runs in internal SRAM
flexspi_nor_debug flexspi_nor_release	The application runs in external flash, which must be programmed/flashed before use. Need extra configuration, see section 5.3 .

- Note that the target has its own set of options so changing for example include path in one target does not affect the other targets.
- To build the demo application, click the “Make” button, highlighted in red below.



5. The build completes without errors.

5.2 Run an Example Application

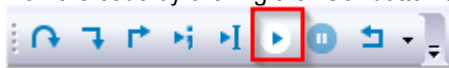
To download and run the application, perform these steps:

1. Connect the debug probe to development platform to your PC via USB cable. See chapter 7 for details how to connect the debug probe.
2. Open the terminal application on the PC, such as TeraTerm or PuTTY, and connect to the debug serial port number. Configure the terminal with 115200 baud, 8N1. You can alter the baud rate by searching for the reference BOARD_DEBUG_UART_BAUDRATE variable in file: **board.h**

3. Click the "Download and Debug" button to download the application to the target.



4. The application is then downloaded to the target and automatically runs to the main() function.
5. Run the code by clicking the "Go" button to start the application.



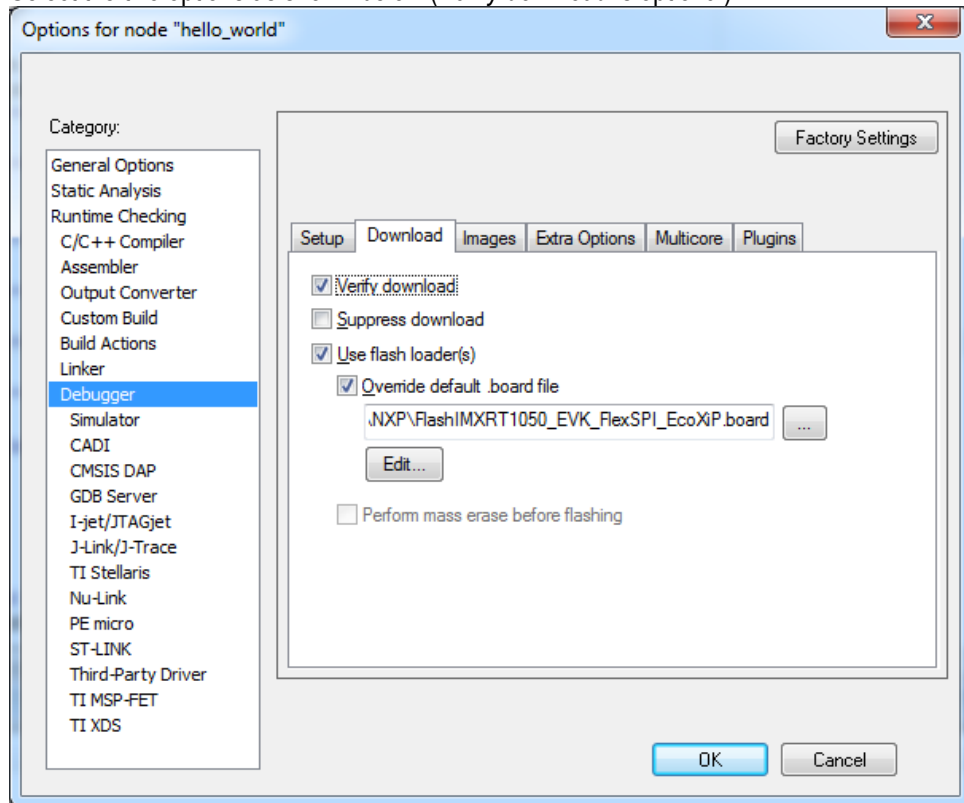
6. The hello_world application is now running and a banner is displayed on the terminal. If this is not true, check your terminal settings and connections.



5.3 Configuring Flash Algorithm

To setup a project for storing/executing the application from the EcoXiP flash:

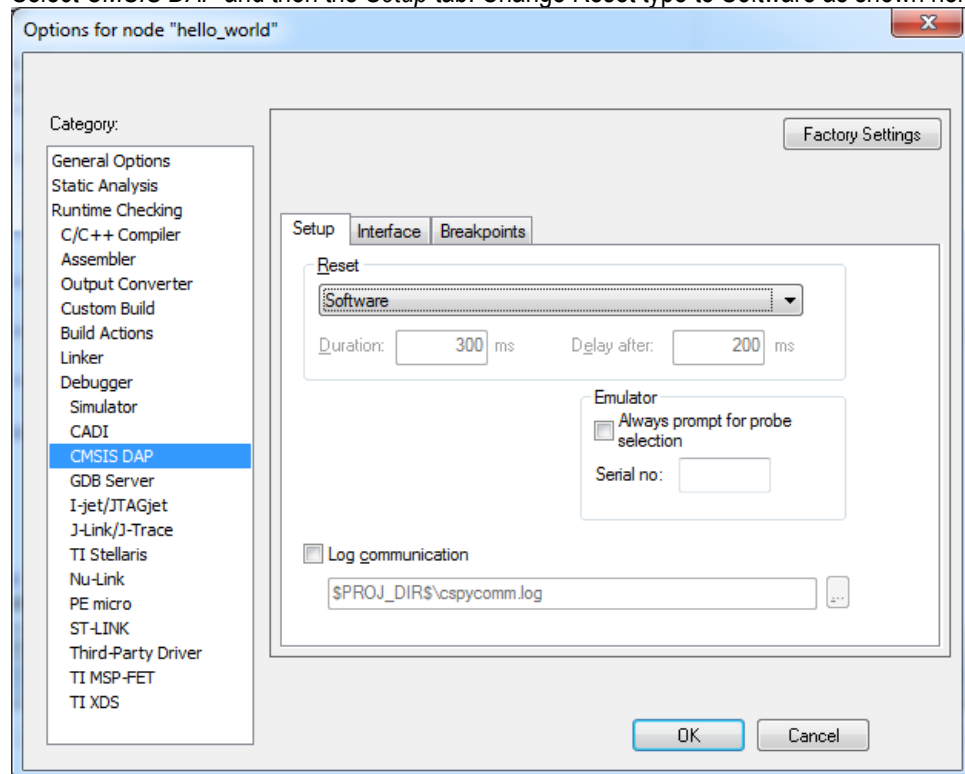
1. Select one of the flexspi_nor_* targets, right click on the project and select *Options* (or press Alt+F7).
2. Select the *Debugger* in the list and then the *Download* tab and select *Settings* for the debugger.
3. Select the two options as shown below (*Verify download* is optional)



The path in the box is
\$TOOLKIT_DIR\$\config\flashloader\NXP\FlexSPI_EcoXiP.board

It is the same path for the iMX RT1052 and the RT1062.

4. Select *CMSIS DAP* and then the *Setup* tab. Change Reset type to Software as shown here:



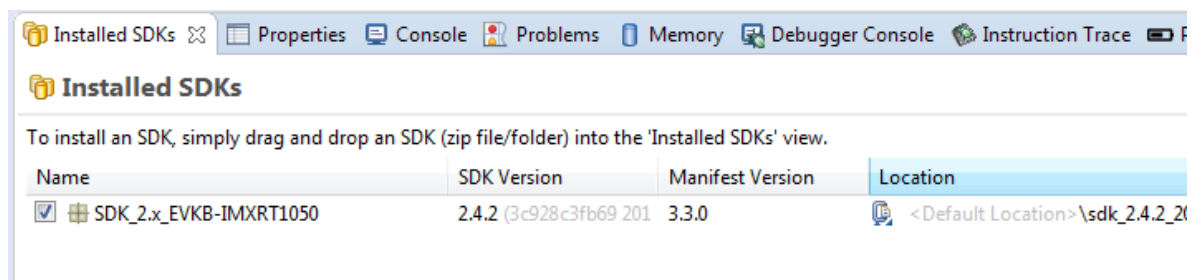
6 Getting Started with NXP MCUXpresso IDE

This section is a guide to open, build and debug an example application using NXP MCUXpresso IDE. It is assumed that you have this development environment installed on your computer.

Make sure to use MCUXpresso version 10.2.1 or later. Older versions did not have support for flash download/programming to the EcoXiP flash memory directly.

6.1 Install the SDK

MCUXpresso requires the SDK to be installed before it can be used. To do that start MCUXpresso and then drag-n-drop the SDK folder (<SDK_Install_Directory>/) that was unpacked and modified in section 2.2 on to the "Installed SDKs" tab in MCUXpresso:

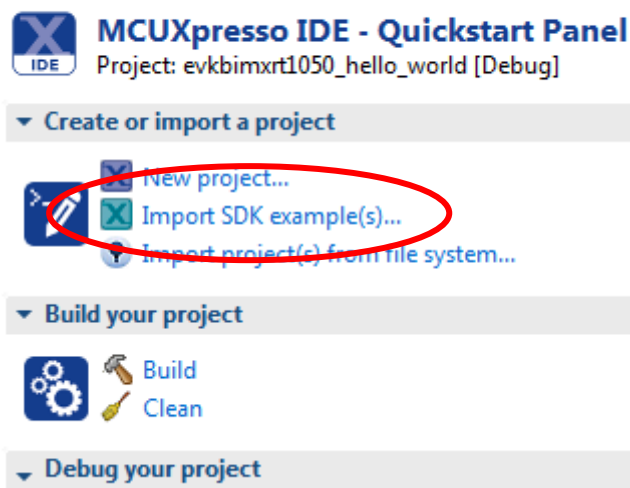
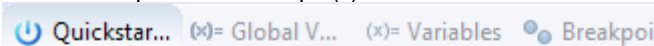


During the installation MCUXpresso will make a copy of the folder that you drag-n-dropped so it is ok to delete the <SDK_Install_Directory>/ afterwards if you don't need it for one of the other IDEs.

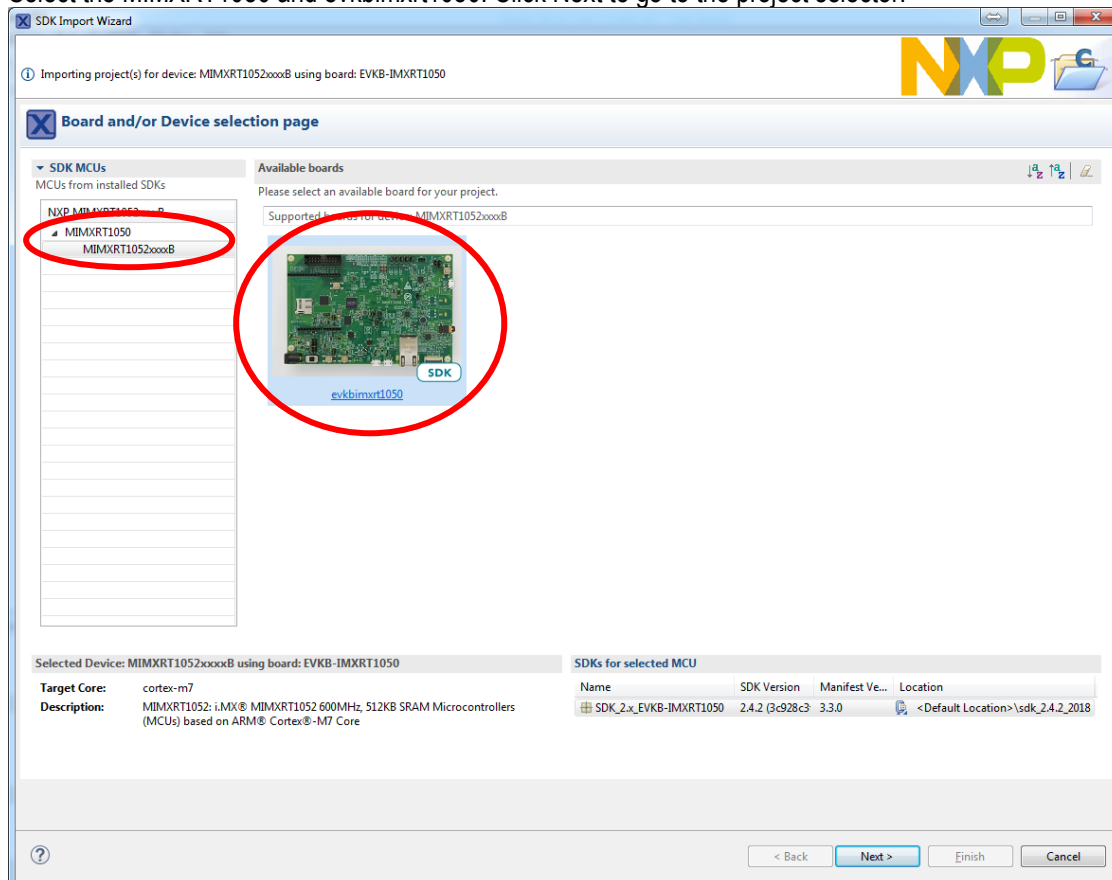
6.2 Build an Example Application

The following steps will guide you through opening the hello_world application from the SDK.

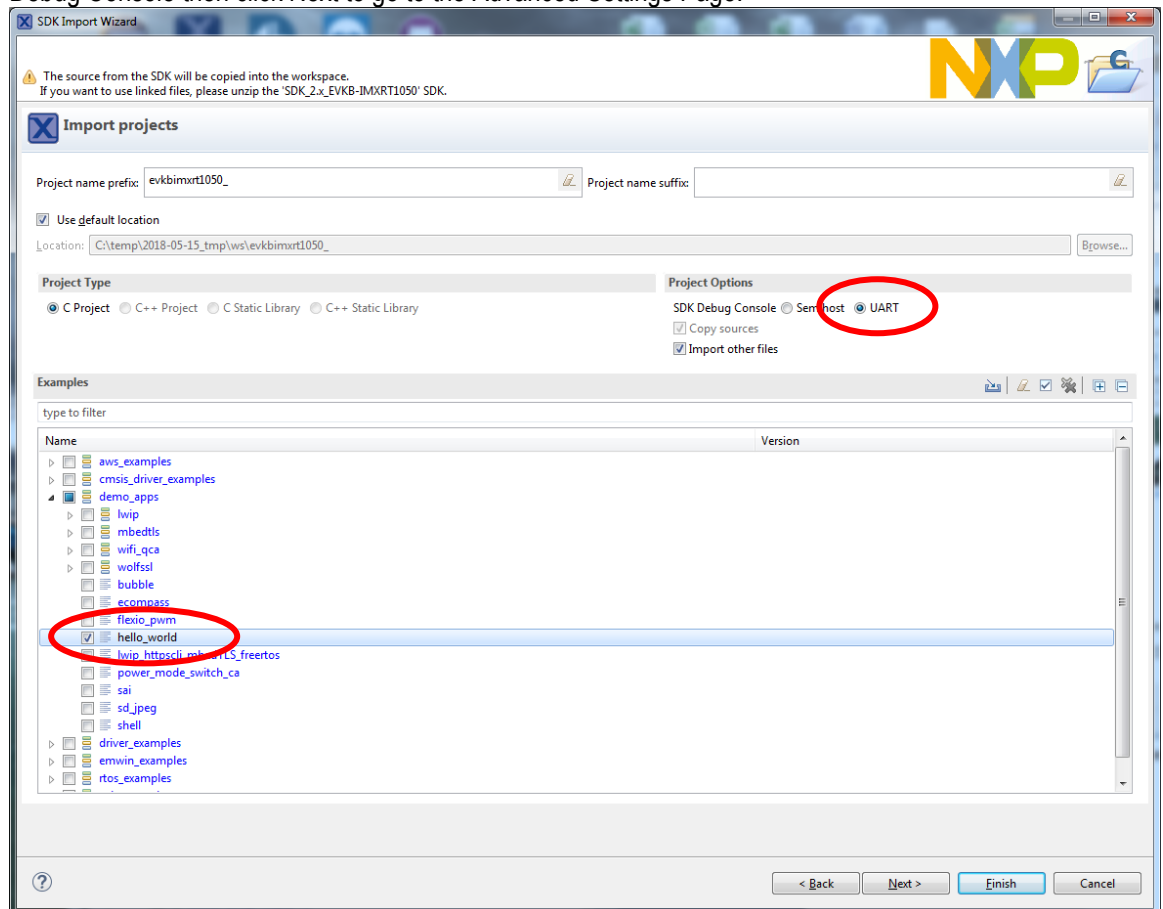
1. Install the SDK as described in section 2.1 if you have not done so already
2. Click the "Import SDK example(s)..." link in the Quickstart Panel



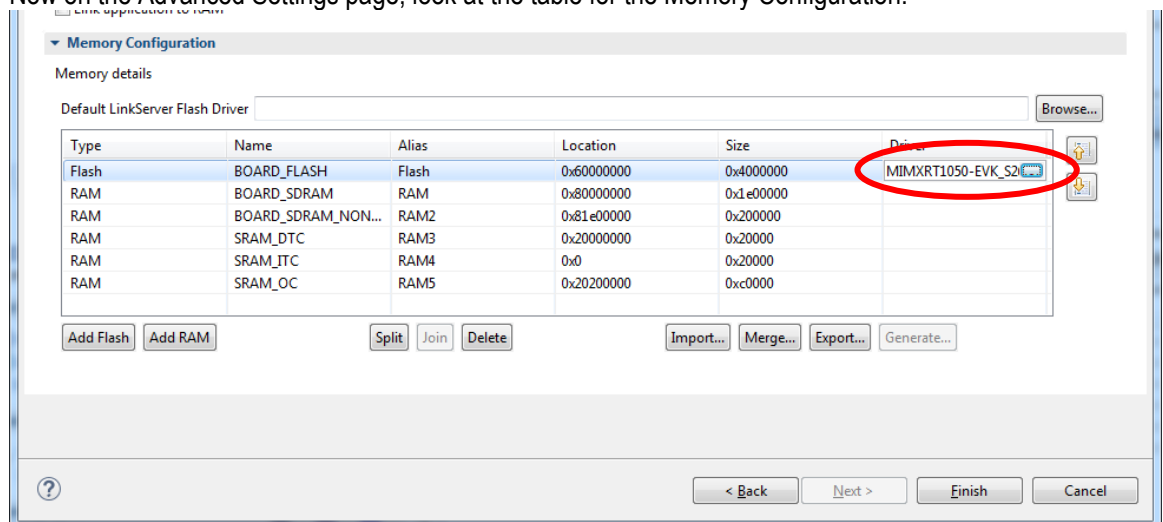
3. Select the MIMXRT1050 and evkbimxrt1050. Click Next to go to the project selector.



- Select the hello_world example and make sure to switch from Semihost to UART for the SDK Debug Console then click Next to go to the Advanced Settings Page.

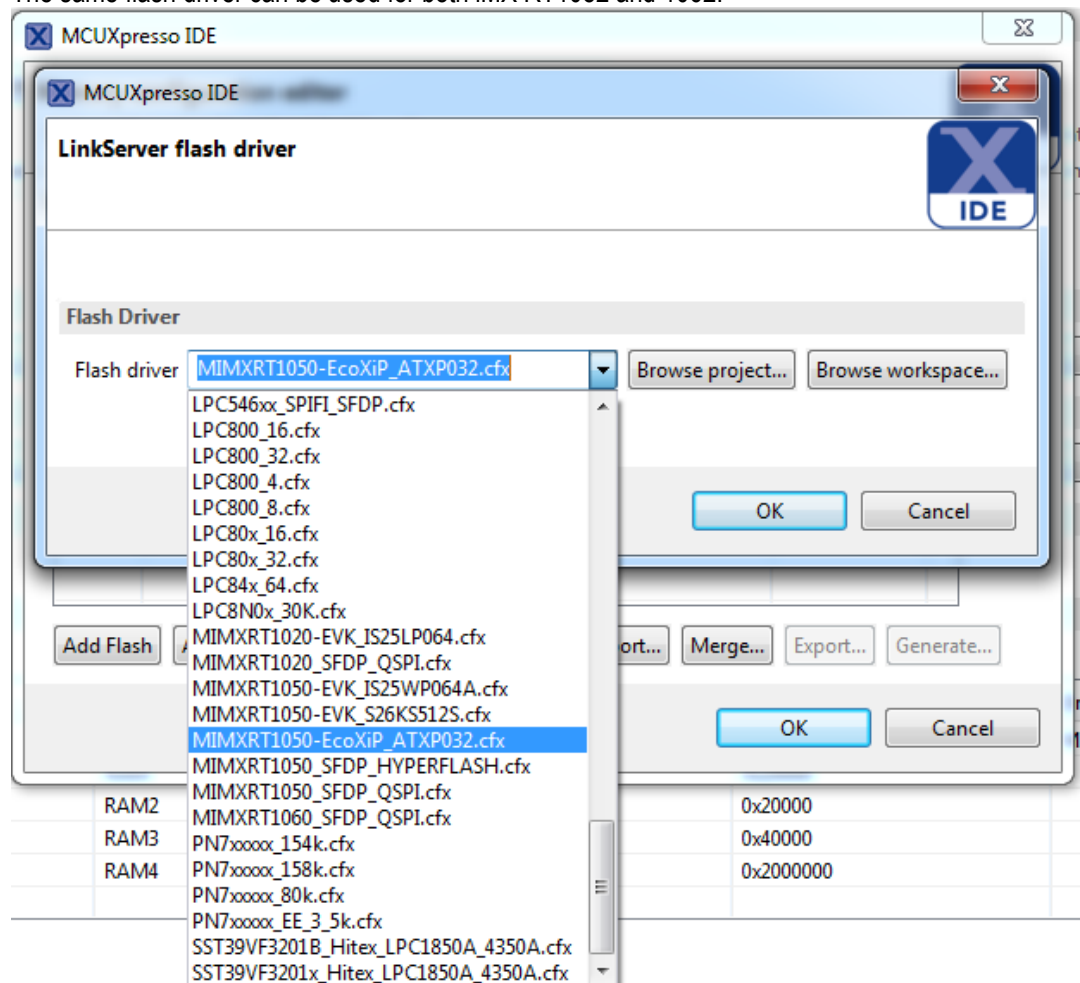


- Now on the Advanced Settings page, look at the table for the Memory Configuration:

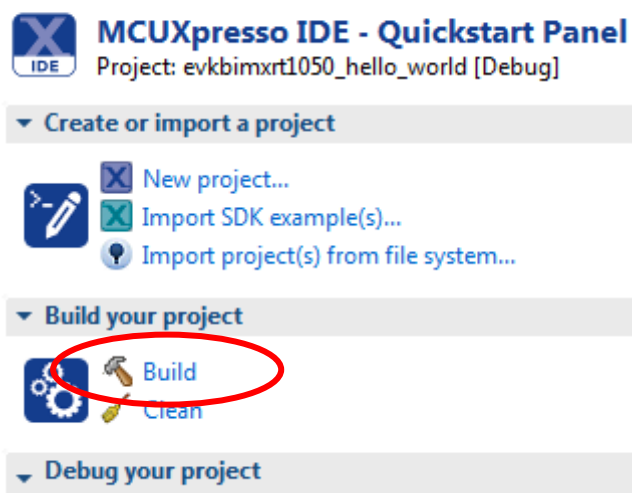
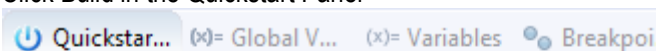


- Click and change the Size for BOARD_FLASH to 0x00400000 (both iMX RT1052 and RT1062 have 4MB flash).
- Click in the table cell for the driver and then on the small button that appears. Change the Driver for BOARD_FLASH from MIMXRT1050-EVK_S26KS512S.cfx to **MIMXRT1050-EcoXiP_ATXP032.cfx** using the dropdown menu.

The same flash driver can be used for both iMX RT1052 and 1062.



8. With all the changes made, click ok and then finish to have MCUXpresso complete the project setup
9. Click Build in the Quickstart Panel



10. The program builds without errors

6.3 Run an Example Application

To download and run the application, perform these steps:

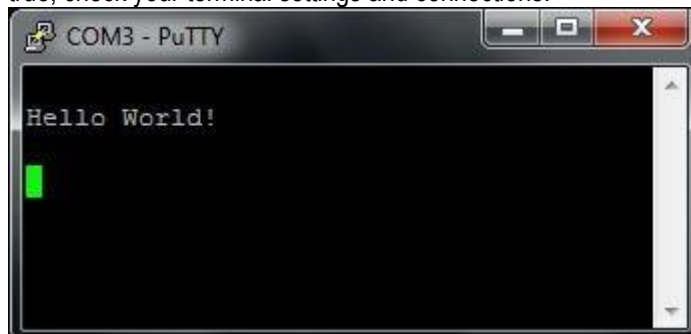
1. Connect the debug probe to development platform to your PC via USB cable. See chapter 7 for details how to connect the debug probe.
2. Open the terminal application on the PC, such as TeraTerm or PuTTY, and connect to the debug serial port number. Configure the terminal with 115200 baud, 8N1. You can alter the baud rate by searching for the reference BOARD_DEBUG_UART_BAUDRATE variable in file: **board.h**
3. Click the "Debug" button in the Quickstart Panel to download the application to the target.



4. The application is then downloaded to the target and automatically runs to the main() function.
5. Run the code by clicking the "Resume" button to start the application.



1. The hello_world application is now running and a banner is displayed on the terminal. If this is not true, check your terminal settings and connections.



7 Debug Interface

It is strongly recommended to use a debug/JTAG probe during program development. The low-cost LPC-Link2 is an excellent choice. Keil ULINK2 and ULINKplus, as well as Segger JLINK, are also excellent debug probes.

There are two debug interface connectors available on the *iMX OEM Carrier board*:

- J10 – this is a Cortex Debug connector. It is a 2x5 pos, 50 mil pitch connector without a shroud. Be careful when inserting the debug probe cable. Position 1 is specifically marked on the PCB silkscreen. It is located in the lower right corner, see Figure 11 below. The connector supports both the SWD and JTAG interfaces.
- J11 – this is an ARM Debug connector. It is a 2x10 pos, 100 mil pitch connector with shroud.

Both connector are defined and supports both the SWD and JTAG type of debug interfaces.

Note that in order to enable the JTAG/SWD interface on the i.MX RT, JP5 shall **not** be shorted/inserted.

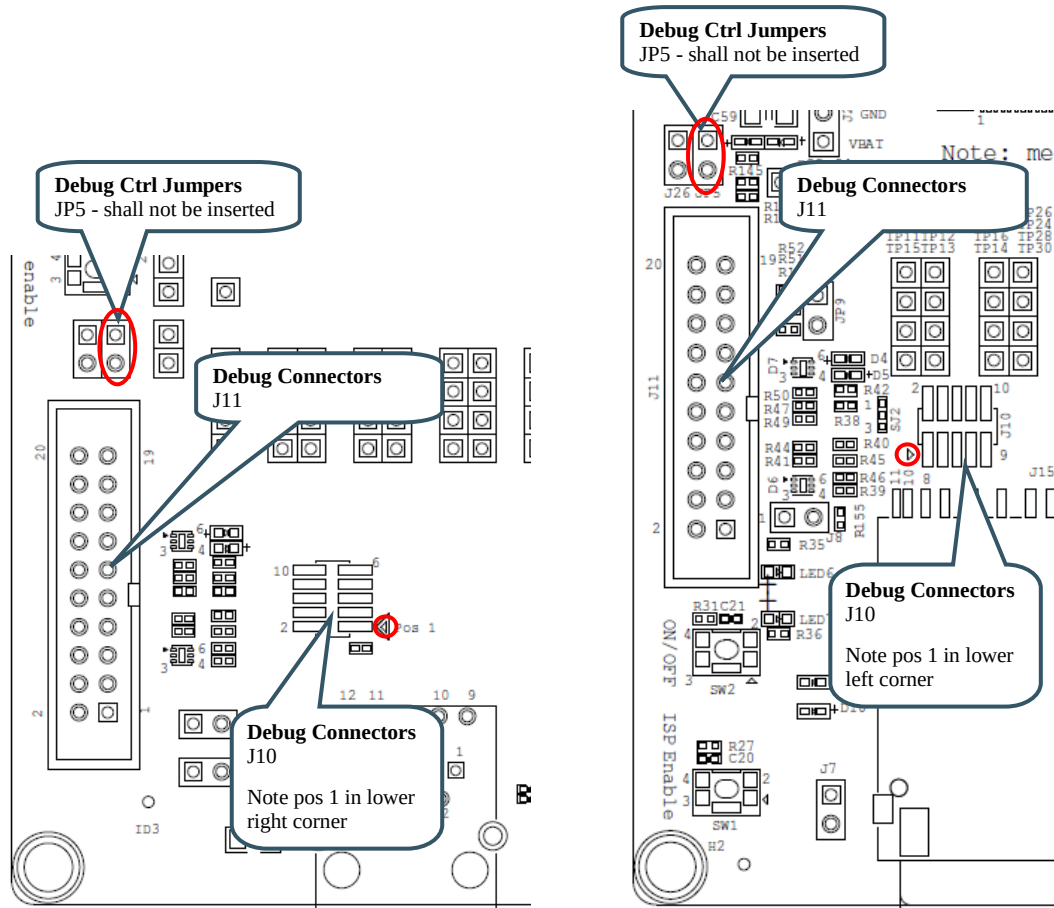


Figure 11 – Debug Interfaces on rev A and rev B1 iMX OEM Carrier board, respectively

Note that due to the powering sequencing requirements on the i.MX RT family, the debug probe I/O voltage **MUST** follow the i.MX RT I/O voltage.

The debug adapter must not drive any output higher than the Vcc/Vref voltage (and if that voltage is zero, then the debug adapter must not drive any output signal). Vcc/Vref is pin 1 on both J10 and J11.

Make sure the debug probe does not have a fixed output voltage, but rather follow Vcc/Vref. If using LPC-Link2 as debug interface, make sure there is **NO** jumper inserted in JP2 on the LPC-Link2.

7.1 J-LINK/J-TRACE Support

This section describes the steps necessary to get the Segger J-TRACE to work with NXP MCUXpresso and Keil uVision. The same instructions are likely to work for Segger J-LINK as well, but it has not been verified.

7.1.1 Install J-LINK Software

The software for the J-TRACE/J-LINK is installed in a central location (i.e. outside of the IDEs). The latest version (v6.42b) as of February 2019 have support for the iMX RT1052/1062 but needs a configuration change to support the EcoXiP flash memory:

1. Download and install the latest version of Segger's software: https://www.segger.com/downloads/jlink/JLink_Windows.exe (v6.42, or later). The rest of the steps will use <jdir> as abbreviation for the folder that the driver was installed in, for example c:\Program Files (x86)\SEGGER\JLink_V642b\
2. Open <jdir>\JLinkDevices.xml
3. Search for MCIMXRT1052 or MCIMXRT1062 to find <Device> entries (at the time this document was written there were MCIMXRT1052, MCIMXRT1052xxxxA and MCIMXRT1052xxxxB, MCIMXRT1062)
4. In each of the <Device> entries, change

```
Loader="Devices/NXP/iMXRT105x/NXP_iMXRT105x_HyperFlash.elf"
or
Loader="Devices/NXP/iMXRT106x/NXP_iMXRT106x_HyperFlash.elf"
```

to

```
Loader="Devices/NXP/iMXRT105x/NXP_iMXRT105x_QSPI.elf"
or
Loader="Devices/NXP/iMXRT106x/NXP_iMXRT106x_QSPI.elf"
```

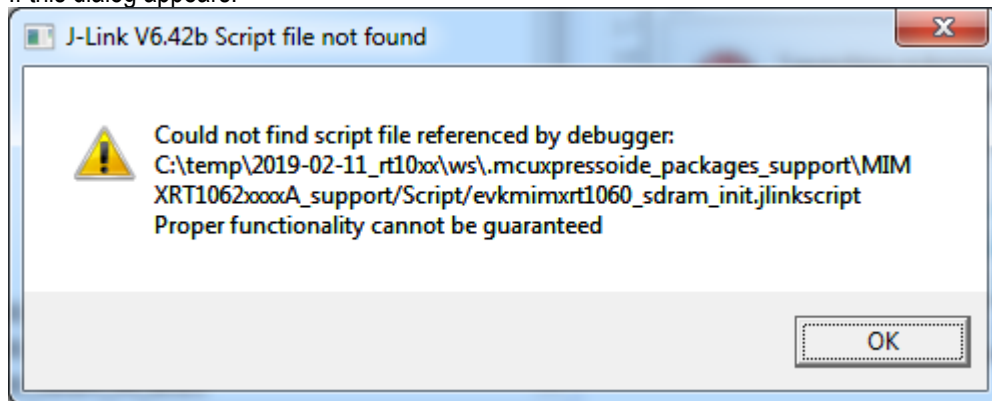
Change the MaxSize="0x04000000" to MaxSize="0x00400000" and then save the file.

7.1.2 MCUXpresso 10.3.0

Chapter 6 describes how to import a project and set it up correctly for the Adesto EcoXiP flash. It is important that the flash algorithm is changed to MIMXRT1050-EcoXiP_ATXP032.cfx and that the flash size is changed to 0x00400000. Note that same flash driver shall be used for both iMX RT1052 and RT1062.

Build and then launch the debugger. MCUXpresso will detect the J-LINK / J-TRACE and configure itself correctly.

If this dialog appears:



Then copy the missing file from the folder that was created in see section 2.2 (c:/temp/ea_files/)

7.1.3 Keil uVision v5.25.2

Chapter 6 describes how to import a project and set it up correctly for the Adesto EcoXip flash. It is important that the flash algorithm is changed to MIMXRT105x EcoXiP Flash and that the flash size is changed to 0x00400000. Note that same flash driver shall be used for both iMX RT1052 and RT1062.

Change the Debugger from the default CMSIS-DAP to J-LINK / J-TRACE Cortex, as shown in picture below:

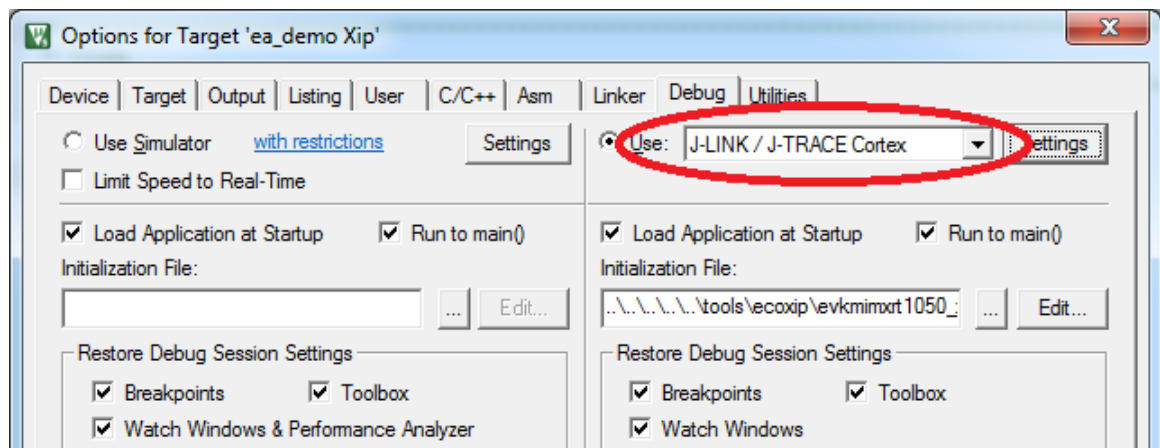


Figure 12 – Setting Debug Interface in Keil uVision

Open the settings dialog and change to the following settings:

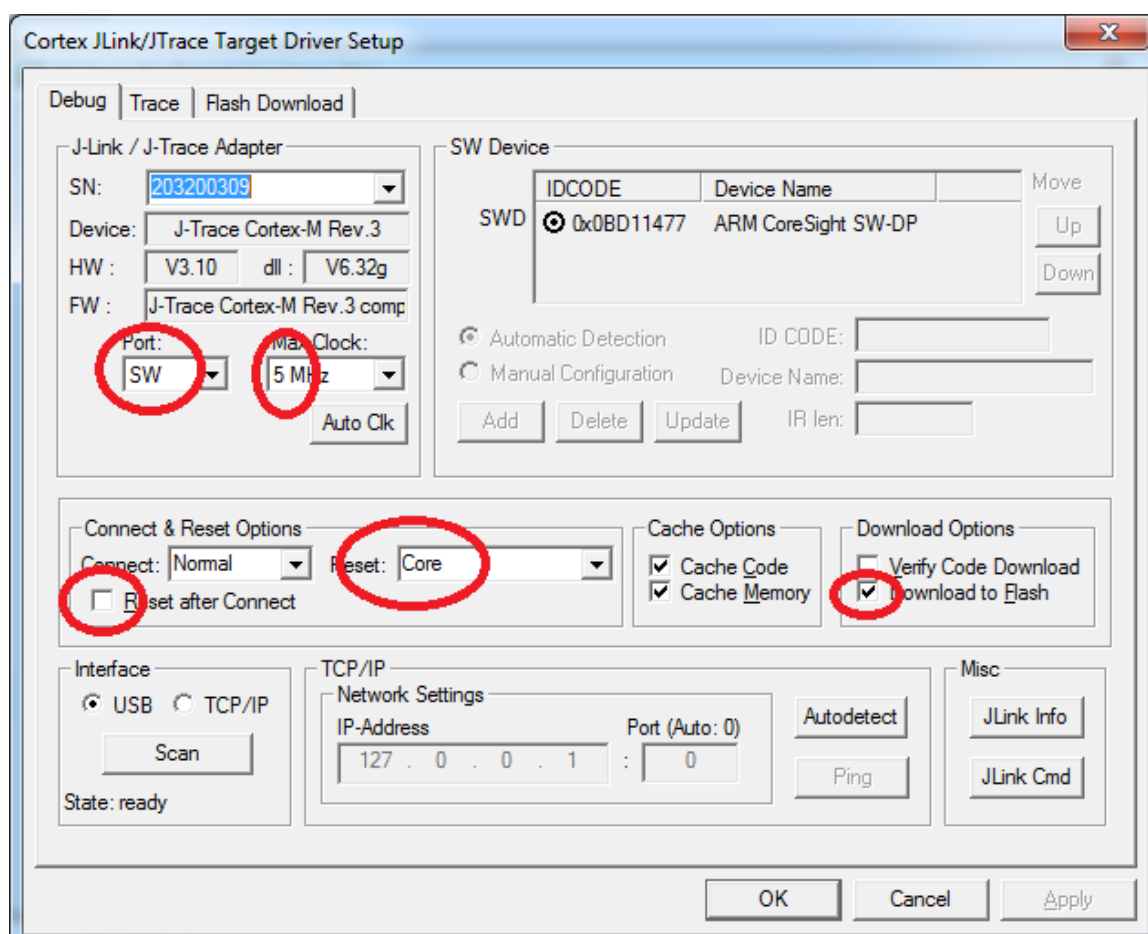


Figure 13 – Configuring J-TRACE/J-LINK Interface in Keil uVision

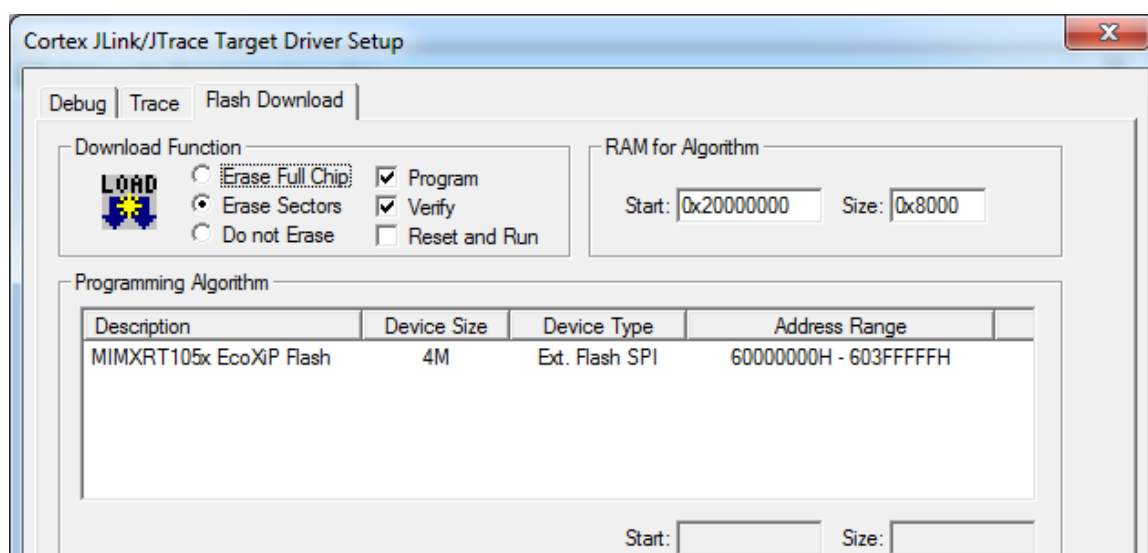


Figure 14 – Configuring Flash Programming for J-TRACE/J-LINK Interface in Keil uVision

7.2 Debug Troubleshooting

In some cases the IDE complains about not being able to connect to the target. This is most likely because the program already running on the target is interfering. The solution is to put the hardware in ISP mode before starting the flash/debug operation in the IDE. To do this:

1. Push and hold down the ISP enable button
2. Press the Reset button
3. Release the Reset button
4. Wait 1 seconds
5. Release the ISP enable button

If the LPC-Link2 debugger is used then there are some additional things to note:

1. Make sure that the J2 jumper on the LPC-Link2 is not inserted. If the jumper is inserted/closed then the target will be powered by the LPC-Link2 which might be too much power for the usb port that the LPC-Link2 is connected to.
2. If the LPC-Link2 is not found by the IDE and you are working on a laptop then try using a powered usb hub instead.
3. The troubleshooting section in this forum post has a couple of additional things to try:
<https://community.nxp.com/thread/388964>
4. There is a *Using and troubleshooting LPC-Link2* in the *Appendix - Additional Hints and Tips* of the User Guide for MCUXpresso IDE. The location of the document is c:\nxp\MCUXpressoIDE_11.0.0_2516\MCUXpresso_IDE_User_Guide.pdf if the IDE was installed with the default settings.

8 Standalone Program Download

This chapter describes how to download an application to the *iMX RT OEM board* without using the IDE. Note that this section does not describe how to create the application code (create the application, compile and link it). It is assumed that a binary file exist that represent the application program.

As a reminder, there are two basic methods for program download:

- **SWD/JTAG Debug Interface**

Using this method, the application can be downloaded to internal SRAM, to external SDRAM or external EcoXiP flash.

This method is tightly integrated with the Integrated Development Environment (IDE) used. Specific scripts (and sometimes flash programming algorithms) must exist for the used IDE. Currently such scripts and drivers exist for Keil uVision/MDK, NXP MCUXpresso and IAR Embedded Workbench. For other IDEs, check supported functions.

- There are many different SWD/JTAG interfaces on the market. NXP has created the low-cost LPC-LINK2, Keil has ULINK2/ULINKpro, Segger has J-LINK, etc.

- **ISP over USB Program Download**

ISP is short for In-System Programming. The i.MX RT MCU contains a bootloader in ROM that can be enabled by pressing the ISP Enable push-button.

An application, Mfgtool2 provided by NXP, is needed on the PC for downloading and flashing the application code. It is this method that will be described in this chapter.

The ISP over USB download is accomplished by using the Mfgtool application running on a PC.

Note the following limitations:

- The Mfgtool application is only available for Windows.
- The Mfgtool application must be downloaded from the NXP website for license reasons.
- The setup requires a couple of small modifications to work with the EcoXiP flash from Adesto. These modifications are explained below.

8.1 Install and Patch the Required Software

First of all, download *MCUXpresso Secure Provisioning Tools* from NXP's website. It can be found under the *Tools & Software* tab for each MCU or directly [here](#). The tool is available for Windows, MacOS and Linux but this document only covers the Windows use case. Patching and using the MacOS and Linux versions should be very similar with only paths and file names varying.

The default installation location is `c:\nxp\MCUX_Provi_v1\` which will be referred to as <install directory> from now on.

There are a couple of things that must be patched in the tool itself before it can be used. Download the zip-file from the *Resources* tab on the product page on Embedded Artists' website. The name of the file is **imxrt_secure_provisioning_<date>.zip**. Unpack the zip file to find two files.

Move `EA_iMXRT10x2_DevKit_EcoXiP.json` to <install directory>\bin\data\boot_devices\devices\flex-spi-nor\

Move `blhost_win_wrapper.bat` to <install directory>\bin\tools_scripts\, replacing the existing file.

The *MCUXpresso Secure Provisioning Tool* is now ready to be used.

8.2 Prepare the Program to Flash

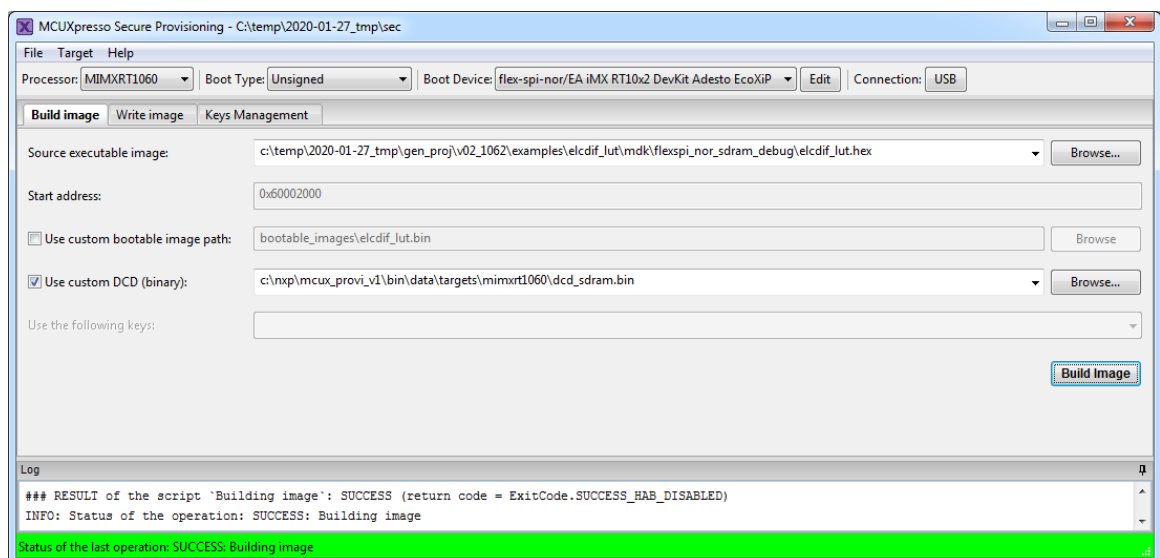
A program can be setup to run directly in external nor flash, in internal RAM or in SDRAM and the project must be modified accordingly. This is described in detail in section 6.3 of the *User Manual - MCUXpresso Secure Provisioning Tools* that comes with the installation of the tool (filename: MCUXpresso Secure Provisioning Tools.pdf) or it can be viewed here: <https://www.nxp.com/docs/en/user-guide/MCUXSPTUG.pdf>.

8.3 Booting an Unsigned Image

Unsigned image is typically used for development. It's recommended to start with this boot type before working with secured images to verify that the executable image works properly.

The first step is to convert the prepared application into a bootable image:

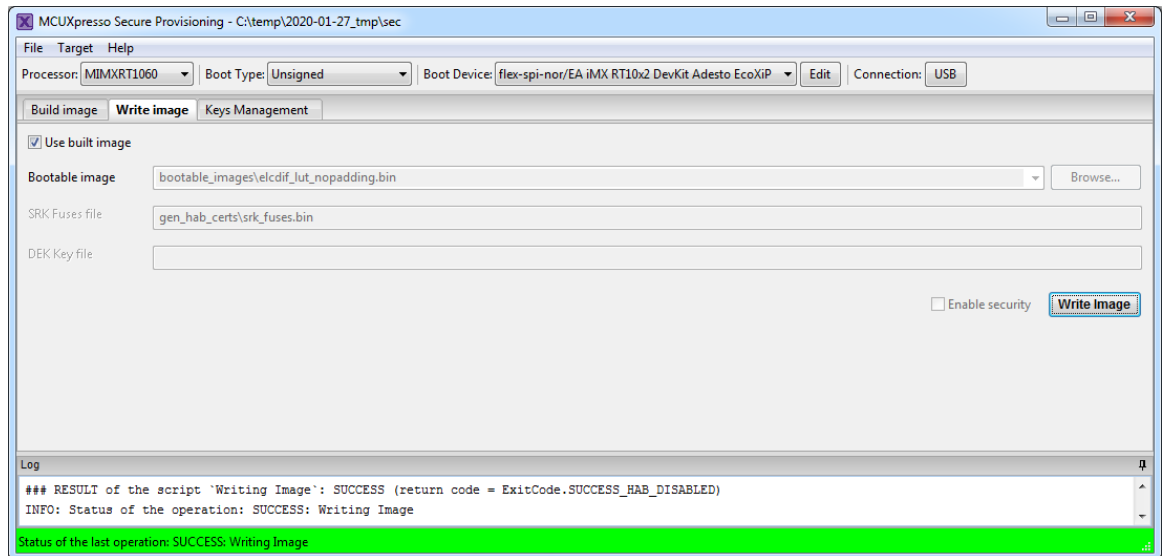
1. Select the correct processor
2. Make sure that Boot Type is *Unsigned*
3. Set Boot Device to *flex-spi-nor/EA iMX RT10x2 DevKit Adesto EcoXiP*
4. Switch to the *Build Image* tab
5. Select the Source executable image that was prepared in the previous section
6. If (and only if) the application uses SDRAM, select the *Use custom DCD* option and point to the `dcd_sdram.bin` file from the `<install directory>\bin\data\targets\mimxrt*` folder
7. Click *Build Image*



To write the image:

1. Switch to the *Write Image* tab
2. Make sure that the *Use built image* option is selected
3. Connect the hardware to the PC using micro USB cables in both J12 and J22 connectors
4. Put the hardware in ISP mode:
 - a. Push and hold down the ISP enable button
 - b. Press the Reset button

- c. Release the Reset button
 - d. Wait 1 seconds
 - e. Release the ISP enable button
5. Press the *Write Image* button
6. When finished, press the Reset button on the hardware to run the program



8.4 Booting an Authenticated or Encrypted Image

The *MCUXpresso Secure Provisioning Tools* support authenticated (signed) and encrypted (signed+encrypted) images. This is described in detail in the User Manual for the tool, in sections 5.8 and 6.7-6.9. A couple of **very important notes**:

1. Encrypted images cannot be used for applications that execute directly in the flash (XiP).
2. Burning the fuses in the processor is an irreversible operation. If the fuses are burned and the key is lost then there is no way to burn anything again on that hardware so make sure to backup the (generated) keys **BEFORE** burning them to the hardware.
3. If the instructions mentions setting the SW7 DIP to the board to 0001 it means putting the board into ISP mode. This is done on the Developers Kit by:
 - a. Push and hold down the ISP enable button
 - b. Press the Reset button
 - c. Release the Reset button
 - d. Wait 1 seconds
 - e. Release the ISP enable button

9 Terminal Application Setup

This chapter contains information about the terminal connection that exist on the *iMX OEM Carrier Board*, and how to setup a terminal application on the PC. The terminal connection connects UART1 of the i.MX RT to a virtual COM port over the USB interface available on J22. The terminal is commonly used during program development.

9.1 UART-to-USB Bridge

The UART-to-USB bridge chip (FT230XS-R from FTDI) on the *iMX OEM Carrier Board* connects to UART channel #1 on the i.MX RT. It exists to simplify connection to a PC because serial ports are not so common any more, especially not on laptops. The USB port also offers the possibility to power the board.

There are two LEDs, transmit from the board (LED11) and receive to the board (LED10), that signal communication activity.

See picture below for locating relevant components.

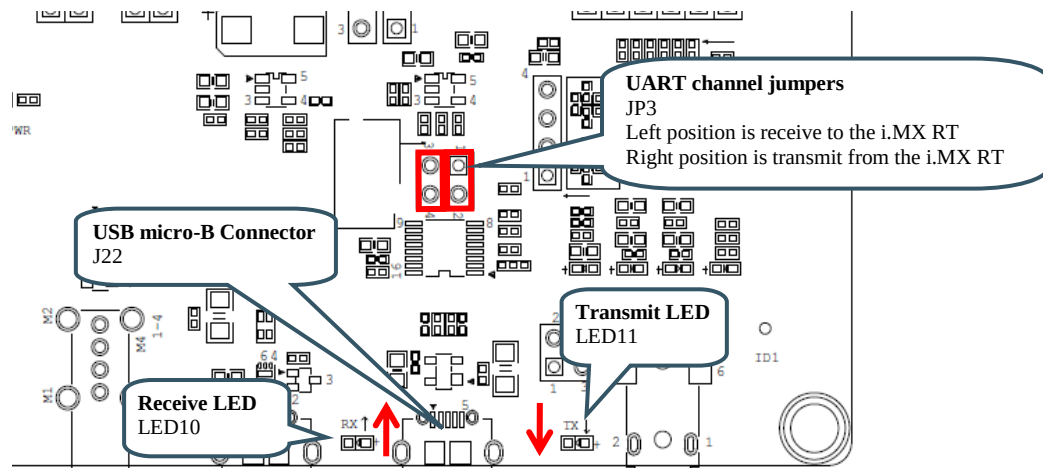


Figure 15 – UART-to-USB Bridge

9.2 Terminal Application on the PC

Begin by connecting the micro-B USB connector to J22, see picture above. Connect the other end of the USB cable to the PC. The PC will typically immediately begin installing drivers automatically for the UART-to-USB bridge that creates a Virtual COM port, if they are not already installed. If you have problems the drivers can be downloaded from the links below:

<http://www.ftdichip.com/Drivers/VCP.htm>

<http://www.ftdichip.com/Support/Documents/InstallGuides.htm>

When the driver has been installed, a new COM port will listed under “Ports” in the Device Manager as shown in Figure 16. Please note that the actual port number will most likely be different on your computer.

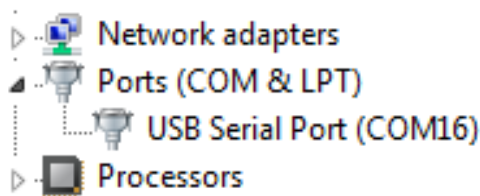


Figure 16 – Virtual COM port shown in device manager

The next step is to open a terminal application and attached it to the Virtual COM port that has just been created. The baud rate should be **115200**.

Some development environments/IDEs have a built-in terminal application that can be used. Sometimes it is better to have a terminal application with more features. For increased flexibility, we recommend using any of the two alternative terminal applications presented in the following sub-sections.

9.2.1 Tera Term Terminal Emulation Application

We recommend that you use **Tera Term** which can be downloaded and installed from either of the links below.

<https://tssh2.osdn.jp/index.html.en>

<http://sourceforge.jp/projects/tssh2/releases/>

Launch *Tera Term*. The first time it launches, it will show you the following dialog. Select the serial option. Assuming the USB cable is connected to the *iMX OEM Carrier Board*, there should be a COM port automatically populated in the list.

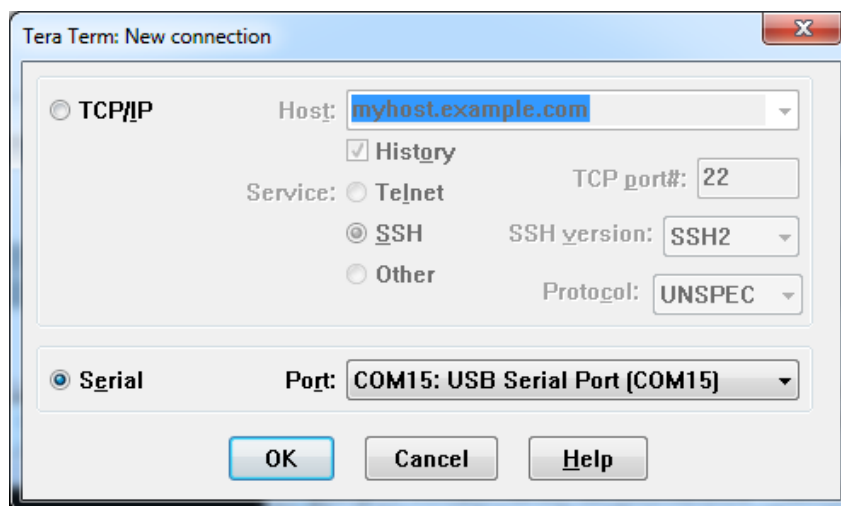


Figure 17 – Tera Term New Connection Window

Configure the serial port settings (using the COM port number identified earlier) to 115200 baud rate, 8 data bits, no parity and 1 stop bit. To do this, go to Setup → Serial Port and change the settings.

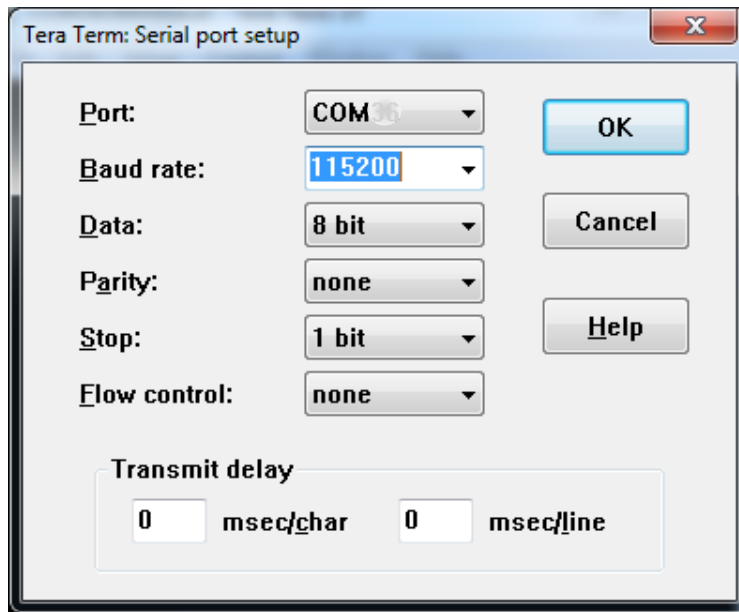


Figure 18 – Tera Term Serial Port Setup

Verify that the connection is open. If connected, *Tera Term* will show something like below in its title bar.

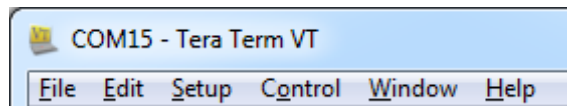


Figure 19 – Tera Term Menu

9.2.2 PuTTY terminal emulation application

Alternatively you can use **PuTTY**. It is another commonly used terminal emulation application. PuTTY can be downloaded and installed from the link below.

<http://www.chiark.greenend.org.uk/~sgtatham/putty/download.html>

Launch PuTTY by either double clicking on the *.exe file you downloaded or from the Start menu, depending on the type of download you selected.

In the window that launches, select the Serial radio button and enter the COM port number that you determined earlier. Also enter the baud rate, in this case 115200.

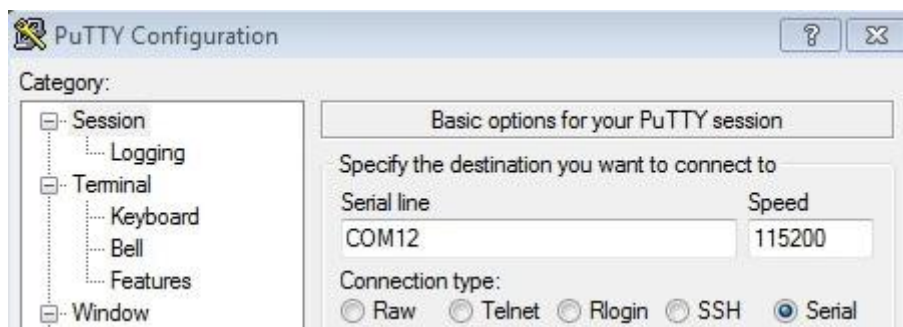


Figure 20 – PuTTY New Session Configuration

Click Open to open the serial connection. Assuming the FTDI cable is connected and you entered the correct COM port, the terminal window will open. If the configuration is not correct, PuTTY will alert you.

10 Booting from External Memory

The i.MX RT does not have any internal flash memory for storing the application. It has to be stored in an external memory. On the *iMX RT OEM board*, this external memory is a 4 MByte OctalSPI, called EcoXiP ATXP032 from Adesto Technologies. The i.MX RT MCU always boots (i.e., starts executing) from this memory.

The EcoXiP memory is a low-power memory with a high-performance interface that operates in DDR (Double Data Rate) mode at 131 MHz. The performance when executing directly from the EcoXiP memory is very good. For more information about this product family, see

<http://www.adestotech.com/products/ecoxip/>

First, let's investigate the three use-cases when executing an application. The picture below illustrates the first main use-case when executing an application.

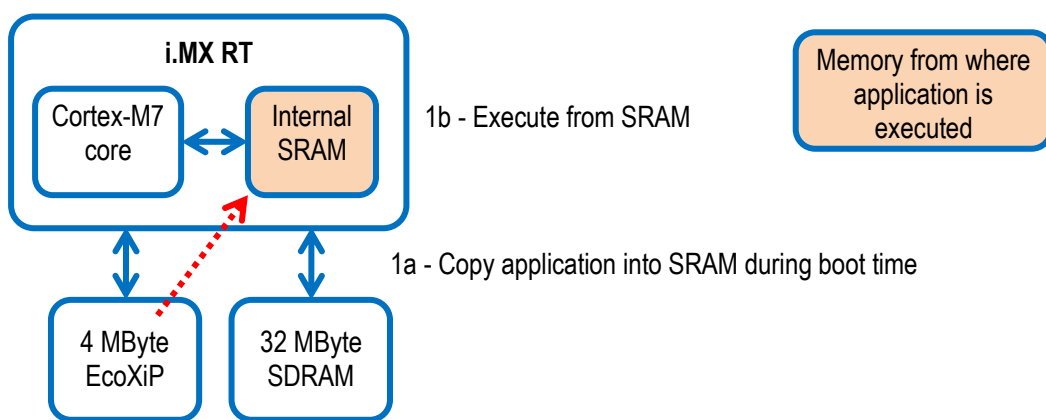


Figure 21 – i.MX RT and EcoXiP - Executing from SRAM

1. The application is stored in the EcoXiP and the bootloader (inside the i.MX RT) copies it into internal SRAM and then run from there.
 - a. The execution performance will be the highest in this setup.
 - b. CoreMark score (about) 2600 / 3000 when running at 528 / 600 MHz
 - c. During program development the application is just downloaded to internal SRAM by the debugger. There is no need to first download the application to the EcoXiP flash memory. The address (in SRAM) where the application is downloaded is the same that it will be copied to by the on-chip bootloader in a final deployed system.

The second use-case is illustrated below. It is the default option supported when compiling and building the Xip targets.

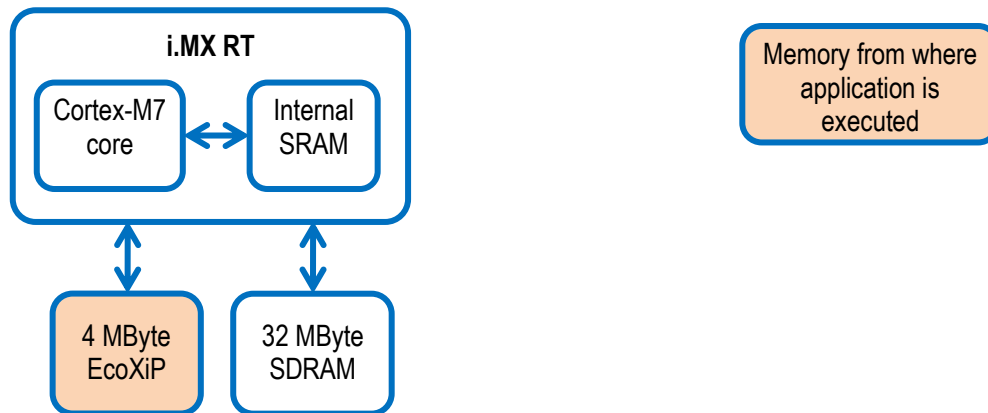


Figure 22 – i.MX RT and EcoXiP - Executing from EcoXiP

2. The application is stored in the EcoXiP and also executed from there. In this case, the internal SRAM is probably too small for the application or is simply used for other things like data storage.
 - a. The execution performance will be about 2/3 of the performance when executing from the EcoXiP (assuming 5% cache miss, which is typical).
 - b. CoreMark score (about) 1736 / 1912 when running at 528 / 600 MHz
 - c. During program development the application must be downloaded/flushed to the EcoXiP memory before debugging actually starts. This is normally handled automatically by the IDE (Integrated Development Environment).

The third use-case is just a mixture of the two main ones. Two, or more memories, are used for storing executable code.

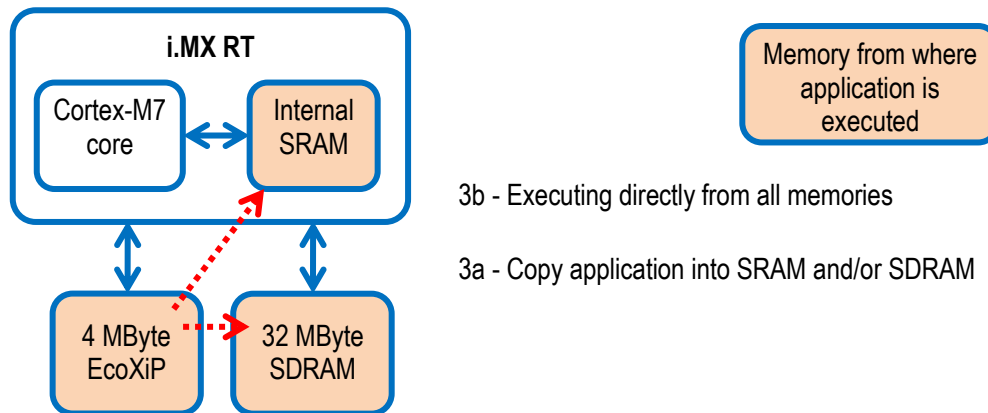


Figure 23 – i.MX RT and EcoXiP - Executing from all memories

3. The third setup is a mixture of the two above. Part of the application is copied into SRAM and/or SDRAM and part is executed directly from the EcoXiP. A reason for placing part of an application in SRAM can be a need for highest performance for a data processing algorithm or a time critical interrupt service routine.
 - a. Note that this is a more complicated system architecture. The application must implement a dynamic loader that can copy code from the EcoXiP to SRAM, either on-demand or in a pre-scheduled way. The linker script can be much more complicated because of this. There is no general solution for this system solution. Every system must be individually investigated in order to select and implement the best solution.

11 Things to Note

11.1 ESD Precaution

Please note that the *iMX RT OEM Board* and *iMX OEM Carrier Board* come without any case/box and all components are exposed for finger touches – and therefore extra attention must be paid to ESD (electrostatic discharge) precaution.

Make it a habit always to first touch the metal surface of one of the USB, SD or Ethernet connectors for a few seconds with both hands before touching any other parts of the boards. That way, you will have the same potential as the board and therefore minimize the risk for ESD.



Never touch directly on the *iMX RT OEM Board* and in general as little as possible on the *iMX OEM Carrier Board*. The push-buttons on the *iMX OEM Carrier Board* have grounded shields to minimize the effect of ESD.

Note that Embedded Artists does not replace boards that have been damaged by ESD.

11.2 General Handling Care

Handle the *iMX RT OEM Board* and *iMX OEM Carrier Board* with care. The boards are not mounted in a protective case/box and are not designed for rough physical handling. Connectors can wear out after excessive use. The *iMX OEM Carrier Board* is designed for prototyping use, and not for integration into an end-product.

For boards with LCD, do not exercise excessive pressure on the LCD glass area. That will damage the display. Also, do not apply pressure on the flex cables connecting the LCD/touch screen. These are relatively sensitive and can be damaged if too much pressure is applied to them.

Note that Embedded Artists does not replace boards where the LCD has been improperly handled.

11.3 OTP Fuse Programming

The i.MX RT MCU has on-chip OTP fuses that can be programmed, see NXP documents *IMXRT1050CEC, .MX RT1050 Crossover Processors for Consumer Products - Data Sheet* and *IMXRT1050RM, i.MX RT1050 Processor Reference Manual* and *IMXRT1060CEC, .MX RT1060 Crossover Processors for Consumer Products - Data Sheet* and *IMXRT1060RM, i.MX RT1060 Processor Reference Manual* for details. Once programmed, there is no possibility to reprogram them.

iMX RT OEM Boards are delivered without any OTP fuse programming. It is completely up to the OEM board user to decide if OTP fuses shall be programmed and in that case, which ones.

Note that Embedded Artists does not replace iMX RT OEM Boards because of wrong OTP programming. It's the user's responsibility to be absolutely certain before OTP programming and not to program the fuses by accident.

11.4 Further Information

The following NXP documents are important reference documents and should be consulted for functional details:

- IMXRT1050CEC, .iMX RT1050 Crossover Processors for Consumer Products - Data Sheet, latest revision
 - IMXRT1050IEC, .iMX RT1050 Crossover Processors for Industrial Products - Data Sheet, latest revision
 - IMXRT1050RM, i.MX RT1050 Processor Reference Manual, latest revision
 - IMXRT1050CE, Chip Errata for the i.MX RT1050, latest revision
Note: It is the user's responsibility to make sure all errata published by the manufacturer are taken note of. The manufacturer's advice should be followed.
 - AN12094, Power consumption and measurement of i.MXRT1050, latest revision
-
- IMXRT1060CEC, i.MX RT1060 Crossover Processors for Consumer Products - Data Sheet, latest revision
 - IMXRT1060IEC, i.MX RT1060 Crossover Processors for Industrial Products - Data Sheet, latest revision
 - IMXRT1060RM, i.MX RT1060 Processor Reference Manual, latest revision
 - IMXRT1060CE, Chip Errata for the i.MX RT1060, latest revision
Note: It is the user's responsibility to make sure all errata published by the manufacturer are taken note of. The manufacturer's advice should be followed.
 - AN12245, Power consumption and measurement of i.MXRT1060, latest revision
 - AN12253, i.MXRT1060 Product Lifetime Usage Estimates, latest revision

12 Disclaimers

Embedded Artists reserves the right to make changes to information published in this document, including, without limitation, specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Customer is responsible for the design and operation of their applications and products using Embedded Artists' products, and Embedded Artists accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the Embedded Artists' product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products. Customer is required to have expertise in electrical engineering and computer engineering for the installation and use of Embedded Artists' products.

Embedded Artists does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using Embedded Artists' products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). Embedded Artists does not accept any liability in this respect.

Embedded Artists does not accept any liability for errata on individual components. Customer is responsible to make sure all errata published by the manufacturer of each component are taken note of. The manufacturer's advice should be followed.

Embedded Artists does not accept any liability and no warranty is given for any unexpected software behavior due to deficient components.

Customer is required to take note of manufacturer's specification of used components, for example MCU, SDRAM and FLASH. Such specifications, if applicable, contains additional information that must be taken note of for the safe and reliable operation. These documents are stored on Embedded Artists' product support page.

All Embedded Artists' products are sold pursuant to Embedded Artists' terms and conditions of sale: http://www.embeddedartists.com/sites/default/files/docs/General_Terms_and_Conditions.pdf

No license, express or implied, by estoppel or otherwise, to any intellectual property rights is granted under this document. If any part of this document refers to any third party products or services it shall not be deemed a license grant by Embedded Artists for the use of such third party products or services, or any intellectual property contained therein or considered as a warranty covering the use in any manner whatsoever of such third party products or services or any intellectual property contained therein.

UNLESS OTHERWISE SET FORTH IN EMBEDDED ARTISTS' TERMS AND CONDITIONS OF SALE EMBEDDED ARTISTS DISCLAIMS ANY EXPRESS OR IMPLIED WARRANTY WITH RESPECT TO THE USE AND/OR SALE OF EMBEDDED ARTISTS PRODUCTS INCLUDING WITHOUT LIMITATION IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE (AND THEIR EQUIVALENTS UNDER THE LAWS OF ANY JURISDICTION), OR INFRINGEMENT OF ANY PATENT, COPYRIGHT OR OTHER INTELLECTUAL PROPERTY RIGHT.

UNLESS EXPRESSLY APPROVED IN WRITING BY THE CEO OF EMBEDDED ARTISTS, PRODUCTS ARE NOT RECOMMENDED, AUTHORIZED OR WARRANTED FOR USE IN MILITARY, AIR CRAFT, SPACE, NUCLEAR, LIFE SAVING, OR LIFE SUSTAINING APPLICATIONS, NOR IN PRODUCTS OR SYSTEMS WHERE FAILURE OR MALFUNCTION MAY RESULT IN PERSONAL INJURY, DEATH, OR SEVERE PROPERTY OR ENVIRONMENTAL DAMAGE.

Resale of Embedded Artists' products with provisions different from the statements and/or technical features set forth in this document shall immediately void any warranty granted by Embedded Artists

for the Embedded Artists' product or service described herein and shall not create or extend in any manner whatsoever, any liability of Embedded Artists.

This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from national authorities.

12.1 Definition of Document Status

Preliminary – The document is a draft version only. The content is still under internal review and subject to formal approval, which may result in modifications or additions. Embedded Artists does not give any representations or warranties as to the accuracy or completeness of information included herein and shall have no liability for the consequences of use of such information. The document is in this state until the product has passed Embedded Artists product qualification tests.

Approved – The information and data provided define the specification of the product as agreed between Embedded Artists and its customer, unless Embedded Artists and customer have explicitly agreed otherwise in writing.